

第一章 引论

1.OS的定义

操作系统是一组控制和管理计算机软硬件资源、合理地作业进行调度以及方便用户使用的程序集合。

2.操作系统的目标

- 方便性：操作系统使计算机更易于使用
- 有效性：以更有效的方式使用计算机资源（提高系统资源利用率、提高系统的吞吐量）
- 可扩展性：允许有效地开发、测试和引进新的功能
- 开放性：实现应用程序的可移植性和互操作性，要求具有统一地开放的环境

3.OS的作用

- OS作为用户与计算机硬件系统之间的接口
- OS作为计算机系统资源的管理者（软硬件资源）
- OS用作扩充机器（实现了对计算机硬件资源的抽象）（覆盖了软件的机器称为扩充机器）

4.单道批处理系统

- 内存中始终仅存在一道作业运行
- 运行结束或出错才自动调另一道作业运行

5.多道批处理系统

- 内存中存放多道作业
- 运行结束或出错，自动调度内存中的另一道作业运行
- A使用IO时，B可以使用CPU计算

特征：多道性、无序性、调度性

优点：提高了资源利用率和吞吐能力

缺点：平均周转时间长、没有交互能力

6.分时操作系统（时间片轮转）

特点：

- 多路性：一个主机与多个终端相连
- 独立性：彼此独立操作、互不干扰
- 及时性：系统能在很短的时间得到回答
- 交互性：能实现人机对话（区别于批处理系统）

单道分时系统（A、B、C轮流执行）

区分前后台的分时系统

7.实时系统

- 及时响应外部事件的请求并在规定时间内完成处理
- 特点：多路性、独立性、交互性、可靠性、及时性

8.现代OS的四个基本特征：

- 并发性
 - 并行：两个或多个事件在同一时刻发生（A在使用CPU、B在使用IO）
 - 并发：两个或多个事件在同一时间间隔内发生（微观上还是有先后）
- 共享性
 - 系统中的资源可供内存中多个并发执行的进程共同使用
 - 互斥共享方式（一段时间内只有一个进程访问）、同时访问方式（宏观同时，微观交替互斥）
- 虚拟性
 - 把一个物理实体映射为多个逻辑上的对应物
 - 时分复用技术、空分复用技术
- 异步性
 - 执行结果不确定，程序不可再现

(并发是最重要的特征，其他特征都以并发为前提)

(并发和共享是操作系统的两个最基本的特征，它们又是互为存在的条件)

9.操作系统的主要功能

- 处理机管理（CPU）
 - 进程控制、进程同步和互斥、进程通信、进程调度
- 存储器管理
 - 内存分配、内存保护、地址映射、内存扩充
- 设备管理
 - 缓冲管理、设备分配、设备驱动
- 文件管理
 - 文件存储空间管理、目录管理、文件的读写管理和保护
- 方便用户使用的用户接口
 - 用户需要的用户命令、应用软件需要的系统调用

10.OS的结构设计

- 无结构：庞杂混乱、调试困难
- 模块化：进程管理模块、存储器模块、文件管理模块.....；模块难划分

- 层次式：上层依赖下层
- 微内核：更小更精炼，开机后常驻内存

第二章 进程管理

针对近两年题目，考虑一下这章会怎么考：

- 会考一道简答，进程、线程，进程线程区别，进程程序区别等
- 会考一道信号量编程大题

(别的应该就没了，够多了)

1.进程的定义

- 进程是程序的一次执行
- 进程是一个程序及其数据在处理机上顺序执行时所发生的活动
- 进程是程序在一个数据集合上运行的过程，它是系统进行资源分配和调度的一个独立单位

引入进程实体后定义为：进程是进程实体的运行过程，是系统进行资源分配和调度的一个独立单位。

2.进程实体

- 程序段、相关的数据段、PCB

3.进程的特征

- 动态性：进程实体的一次执行过程，有一定的生命期
- 并发性：多个进程同存于内存中，并且能在一段时间内同时运行
- 独立性：进程实体是能独立运行、独立分配资源、独立接受调度的基本单位
- 异步性：进程按各自独立的、不可预知的速度向前推进

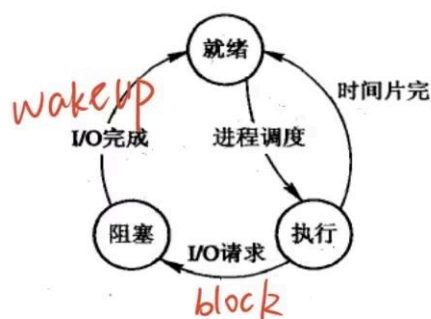
4.进程的三种基本状态



2.2 进程的描述

2.2.2 进程的基本状态及转换

1. 进程的三种基本状态



1) 就绪状态 (Ready)：当进程已分配到除CPU以外的所有必要资源后，只要再获得CPU，便可立即执行

2) 执行状态 (Running)：进程已获得CPU，其程序正在执行

3) 阻塞状态 (Blocked)：正在执行的进程由于发生某事件而暂时无法继续执行时，便放弃处理机而处于暂停状态，把这种暂停状态称为阻塞状态，有时也称为等待状态

电子科技大学

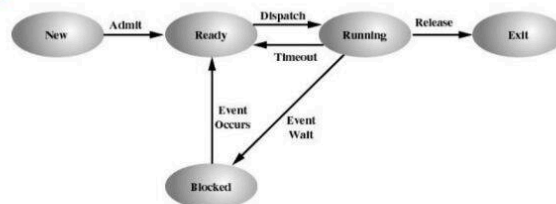
5.进程的五种状态



2.2 进程的描述

2.2.2 进程的基本状态及转换

2. 进程五种状态及转换模型



■ Ready: 准备执行

■ Running: 占用处理机（单处理机环境中，某一时刻仅一个进程占用处理机）

■ Blocked: 等待某事件发生才能执行，如等待I/O完成等

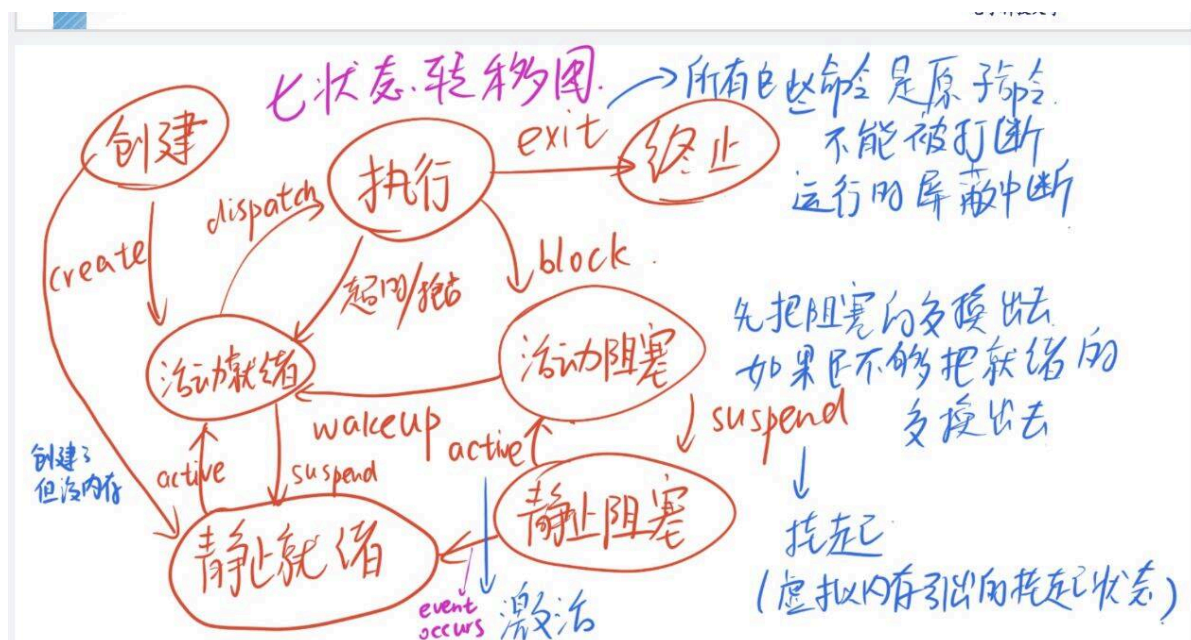
■ New: 进程已经创建，但未被OS接纳为可执行进程，并且程序还在辅存，PCB在内存

■ Exit: 因停止或取消，被OS从执行状态释放

PCB在之后OS检查后销毁

电子科技大学

6. 进程的七状态转移图



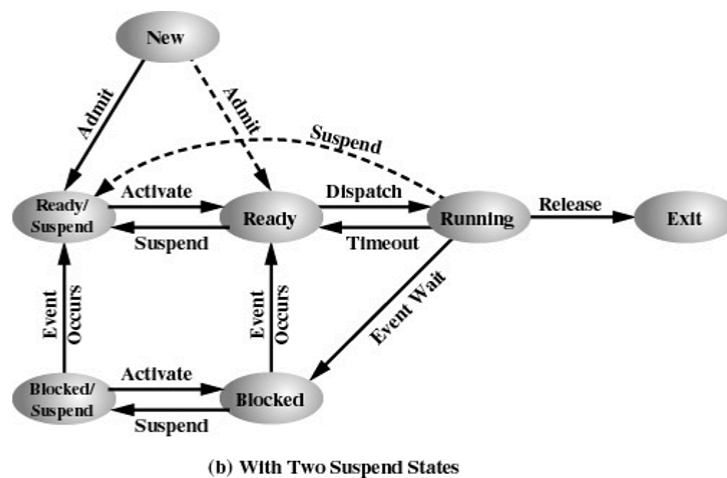
交换到外存，叫“挂起”；区别于“阻塞”

- 进程是否等待事件，阻塞与否
- 进程是否被换出内存，挂起与否
- suspend用于挂起，active相反，用于“静止阻塞→活动阻塞”、“静止就绪→活动就绪”

- Suspend {
- (1) 活动就绪 (Readya) → 静止就绪 (Readys) 。
 - (2) 活动阻塞 (Blockeda) → 静止阻塞 (Blockedys) 。
- Active {
- (3) 静止就绪→活动就绪。
 - (4) 静止阻塞→活动阻塞。

7.进程状态转换

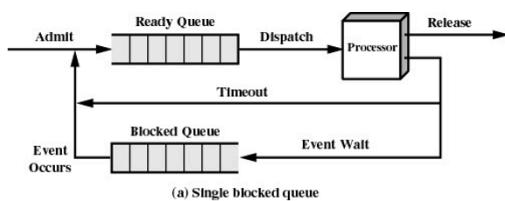
(3) 进程挂起状态的转换



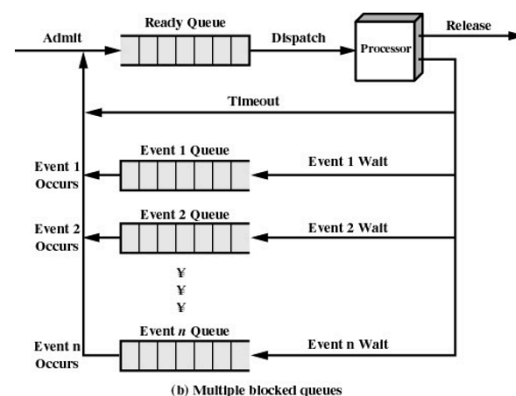
8.单阻塞队列和多阻塞队列

2. 进程五种状态及转换模型

单阻塞队列:



多阻塞队列:



9.内存资源紧张，或者无就绪进程导致处理机空闲的解决办法

- 采用交换技术：换出一部分进程到外存，以腾出内存空间
- 采用虚拟存储技术：每个进程只能装入一部分程序和数据

10.进程控制块PCB

PCB作用：

- 进程存在的唯一标识
- PCB常驻内存

PCB中的信息：

- 进程标识符
- 处理机状态（保护现场）
- 进程调度信息（进程状态、优先级等）
- 进程控制信息（地址、资源清单等）

PCB的组织方式：

- 线性方式、链接方式、索引方式

2.3进程控制

1.进程控制一般是由OS内核中的**原语**来实现的

2.**原子操作**：在一个操作中的所有动作，要么全做，要么全不做

3.处理机的执行状态：

- 系统态，又称管态，也称内核态；（具有较高特权）
- 用户态，又称目态（具有较低特权）

2.4进程同步

1.**进程同步的主要任务**：使并发执行的诸进程之间能有效地共享资源和相互合作，从而使程序的执行具有可再现性

2.临界资源：一次仅允许一个进程访问的资源

3.**临界区**：每个进程中访问临界资源的那部分代码称为临界区

4.同步机制应该遵循的原则

- 空闲让进
- 忙则等待
- 有限等待
- 让权等待

5.软件方法的特点：

- 软件方法始终不能解决“忙等”现象
- 采用软件方法很难控制多个进程的互斥
- 算法设计需要非常小心，否则可能出现死锁，或互斥失败等严重问题

6.信号量：整形信号量、记录型信号量、AND型信号量、信号量集

7.记录型信号量

- 不存在忙等
- Semaphore
- wait (S) 、Signal (S)

8.AND型信号量

- 要么全部分配到进程，要么一个也不分配
- Swait (S1, S2, ..., Sn)
- Ssignal (S1, S2, ..., Sn)

9.信号量的应用

- 利用信号量实现进程互斥
- 利用信号量实现前驱关系

2.5经典进程同步问题

1.生产者——消费者问题

2.哲学家就餐问题

3.读者——写者问题

2.6进程通信

1.进程通信的类型：

- 共享存储器系统（共享数据结构、共享存储区）
- 消息传递系统（直接通信，间接通信）
- 管道通信（连接一个读进程和一个写进程之间通信的共享文件）

2.消息传递通信的实现方法：

- 直接通信方式，利用通信命令
- 间接通信方式，信箱
 - 私用信箱
 - 公用信箱
 - 共享信箱

2.7 线程与线程控制

1.进程体现出两个特点：资源、调度执行

线程：进程内的独立执行代码的实体和调度单元

2.线程的属性

- 轻型实体（线程只拥有少量的必不可少的资源）
- 独立调度和分派的基本单位
- 可并发执行
- 共享进程资源（与其他线程）

3.线程与进程的比较

- 线程具有许多传统进程所具有的特征，所以又称为轻型进程
- 相应地把传统进程称为重型进程
- 传统进程相当于只有一个线程的任务
- 在引入了线程的OS中，每个进程至少有一个线程

调度：

- 不同于传统的OS，在引入了线程的操作系统中：
- 线程作为调度和分派的基本单位
- 进程作为资源拥有的基本单位

并发性：

- 不仅进程之间可以并发执行
- 在一个进程中的多个线程之间亦可并发执行

拥有资源：

- 线程自己不拥有系统资源(也有一点必不可少的资源)
- 但它可以访问其隶属进程的资源

独立性：

- 同一进程中的不同线程共享进程的内存空间和资源
- 同一进程中的不同线程的独立性低于不同进程

系统开销：

- 线程的切换只需要保存和设置少量的寄存器内容，不涉及存储器管理方面的操作
- 由于一个进程中的多个线程具有相同的地址空间，在同步和通信的实现方面线程也比进程容易

支持多处理机系统：

- 一个进程分为多个线程分配到多个处理机上并行执行，可加速进程的完成

4.线程的三种状态

- 执行状态
- 就绪状态
- 阻塞状态

（线程中的挂起状态是指线程的“不可运行状态”，即阻塞状态）

5.线程的三种同步机制

- 互斥锁
- 条件变量
- 信号量机制
 - 私用信号量：用于同一进程中各线程之间的同步，属于特定的进程所有
 - 公用信号量：用于不同进程之间或者不同进程中各线程之间的同步

6.线程实现方式

- 用户级线程：与内核无关，创建、撤销、线程间同步与通信均无需内核来实现
 - 切换不调用内核
 - 调度算法可以在每个进程中灵活制定
 - 可以在一个不支持线程的OS上实现
 - 多线程应用不能利用多处理机进行多重处理
 - 当线程执行一个系统调用时，该线程及其所属进程内的所有线程都会被阻塞
- 内核支持线程：在内核的支持下运行，创建、撤销、切换等都依靠内核实现
 - 如果进程中的一个线程被阻塞了，内核可以调度该进程中的其它线程占有处理器运行，也可以运行其它进程中的线程；
- 组合方式（用户级线程和内核支持线程并存）

7.采用线程的优点：

- 在一个已有进程中创建一个新线程比创建一个全新进程所需的时间少
- 终止一个线程比终止一个进程花费的时间少
- 线程间切换比进程间切换花费的时间少
- 线程提高了不同的执行程序间通信的效率。同一个进程中的线程共享存储空间和文件，它们无需调用内核就可以互相通信。

第三章 处理机调度与死锁

针对近两年题目，考虑一下这章会怎么考：

- 会考一道进程调度大题
- 会考一道银行家算法大题

1.调度的对象：作业、进程

2.三级调度

- 高级调度：**作业调度**或长程调度，用于决定把外存上处于后备队列中的哪些资源调入内存，并为他们创建进程、分配必要的资源，然后，再将新创建的进程排在就绪队列上，准备执行
- 低级调度：**进程调度**或短程调度，用于决定就绪队列中的哪个进程获得处理机，然后再由分派程序把处理机分配给该进程
- 中级调度：中程调度，**就绪状态**←→**挂起状态**，提高内存的利用率和系统吞吐量

3.选择调度算法的准则

调度方式和算法的选择很大程度上取决于操作系统的类型及其目标

调度的目标：

- 提高处理机的利用率
- 提高系统的吞吐量
- 尽量减少进程的等待时间
- 防止进程长期得不到执行

调度的原则：

- 满足用户的需要和系统的需要

面向用户的准则：

- 周转时间短
- 响应时间快
- 截止时间的保证
- 优先权准则

面向系统的准则：

- 系统吞吐量高
- 处理机利用率好
- 各类资源的平衡利用（内存、外存、IO）

4.调度算法

- 先来先服务，FCFS
- 短进程优先，SJF
- 时间片轮转，RR
- 基于优先级的调度算法，PSA
- 剩余时间最短者优先
- 高响应比优先，HRRN
优先权 = $1 + \text{等待时间} / \text{服务时间}$
响应时间 = 等待时间 + 服务时间
- 多级反馈队列调度法

5.调度算法指标

- 周转时间 = 结束时间 - 到达时间
- 带权周转时间 = 周转时间 / 运行时间

6.时间片大小合适：略大于一次典型的交互所需的时间，使大多数交互式进程能在一个时间片内完成

7.两种优先级调度算法（把处理机分配给就绪队列中优先级最高的进程）

- 非抢占式优先权算法（一旦分配了处理机会一直执行下去，直至完成）
- 抢占式优先权调度算法（高优先级任务就绪后，可以抢占正在运行的低优先级任务的处理器）

8.优先权的类型

- 静态优先权（保持不变）
- 动态优先权（响应比）

9.多级反馈队列调度算法

- 第一个队列的优先级最高、第二个次之，优先级越高的队列时间片越小
- 每个队列采用FCFS
- 最低优先权队列采用RR方式运行

实时调度

(P61可以看一眼)

10.常见的两种实时调度算法：

- 最早截止时间优先算法（EDF）
 - 把截止时间最早的排在最前面
 - 可以抢占正在执行的
- 最低松弛度优先算法（LLF）
 - 松弛度 = 完成截止时间 - 当前时间 - 剩余运行时间
 - 松弛度最小的任务排在队列最前面
 - 调度程序选择队首的执行
 - 要么是执行完了触发切换，要么是松弛度为0触发切换（松弛度为0才抢占）

死锁

定义：多个进程在运行过程中因争夺资源而造成的一种僵局

（如果一组进程中的每一个进程都在等待仅由该组进程中的其他进程才能引发的事件，那么该组进程是死锁的。）

1.死锁产生的原因：

- 竞争资源
- 进程间推进顺序非法

2.死锁发生的四个必要条件

- **互斥条件：**进程对所分配到的资源进行排他性使用
- **请求和保持条件：**进程已经保持了至少一个资源，但又提出了新的资源请求
- **不剥夺条件：**进程已获得的资源，在未使用完之前不能被剥夺，只能在使用完时由自己释放
- **循环等待条件：**指在发生死锁时，必然存在一个进程——资源的环形链

3.解决死锁的方法：

- 预防死锁：通过设置限制条件破坏死锁的必要条件
- 避免死锁：在分配资源的过程中，避免进入不安全状态
- 检测死锁：检测到死锁的发生，并定位到与死锁有关的进程和资源

- 解除死锁：检测到之后，常撤销或挂起一些进程

4.预防死锁：使四个必要条件中2、3、4之一不能成立；1不能破坏，还应该加以保证

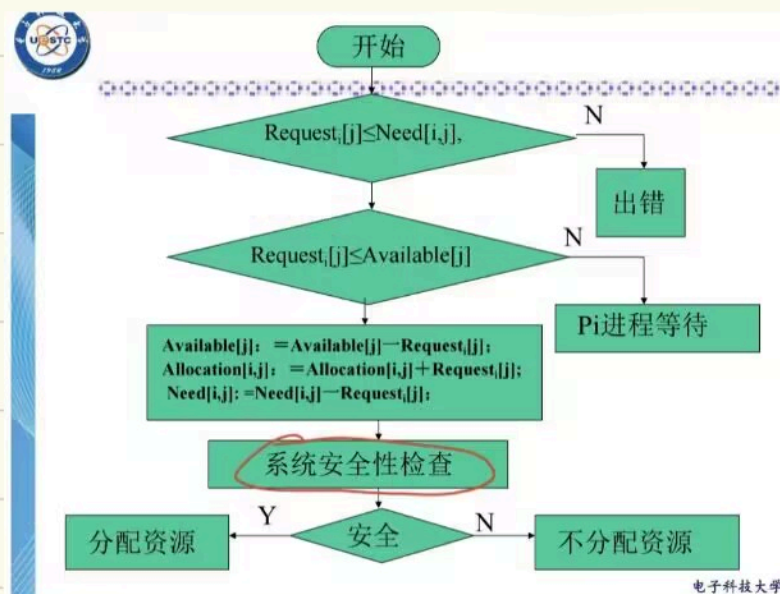
破坏4：对资源按类型编号，所有进程对资源的请求必须严格按照资源序号递增的次序提出

5.避免死锁，能找到安全序列 $\langle p_1, p_2, \dots, p_n \rangle$

6.银行家算法

数据结构：

数据结构：
 $available$ m 个元素的数组 现有资源
 Max $n \times m$ 的矩阵 最大需求
 $Allocation$ $n \times m$ 矩阵 已分配资源
 $Need$ $n \times m$ 矩阵 还需要多少个才能完成任务
 $Need[i, j] = Max[i, j] - Allocation[i, j]$



安全性检查：

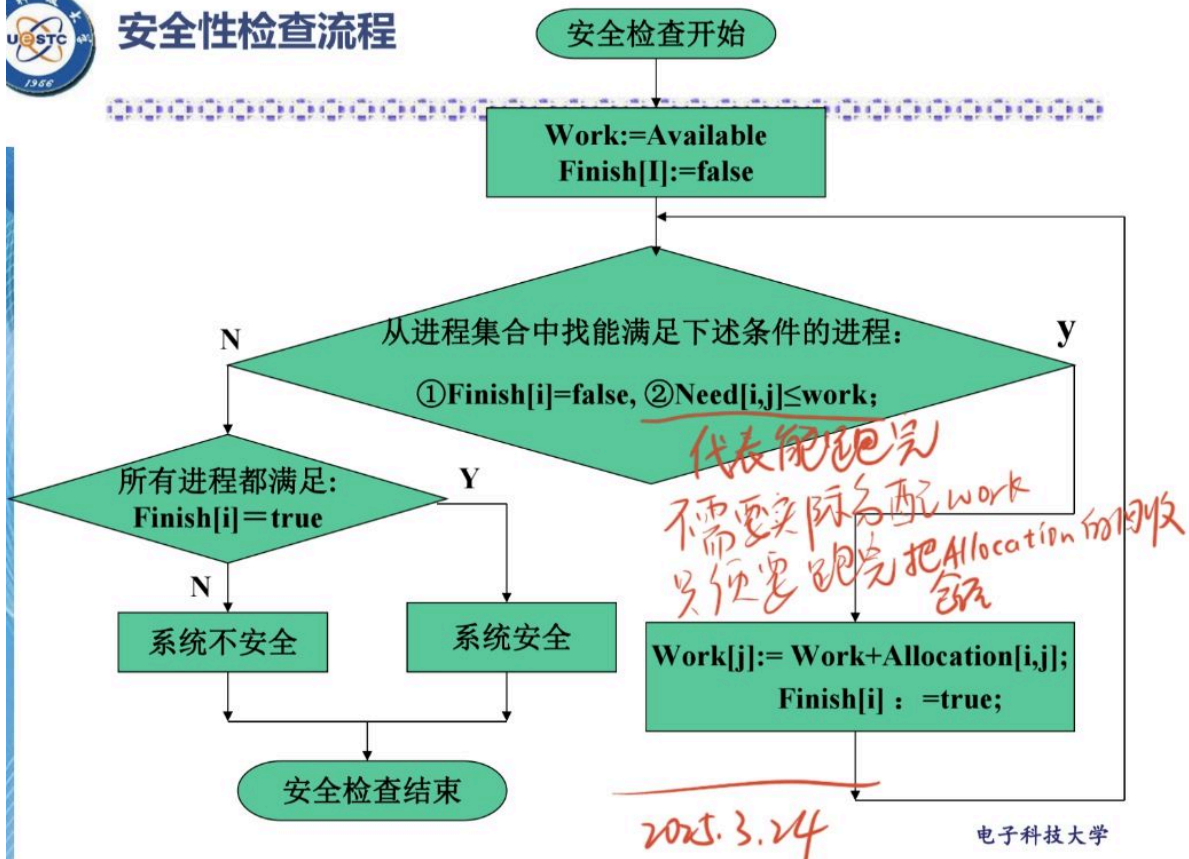
两个向量：① $work$ 表示可提供的资源，初始值 = $available$

② $Finish[i]$ 初始值 $Finish[i] = false$
 当 $finish[i] = true$ 表示 P_i 可执行完毕

安全性检查流程：



安全性检查流程



7. n 个进程共享 m 个同种资源，如果每个进程都需要使用该资源，而且各进程对该类资源的最大需求量之和小于 $m+n$ ，说明该系统不会因竞争该类资源而死锁（P133）

8.死锁的检测:

资源分配图：指向 p_1 的，是资源分配边； p_1 指向的，是资源请求边

选一个既不阻塞又非独立的结点 p ，把资源分配给他，执行完，移除，直至所有结点移除

可以证明：当且仅当系统某状态 S 所对应的资源分配图是不可完全化简的， S 是死锁状态

9.死锁的解除

- 两种方法：剥夺资源、撤销进程
- 付出代价最小的死锁解除算法

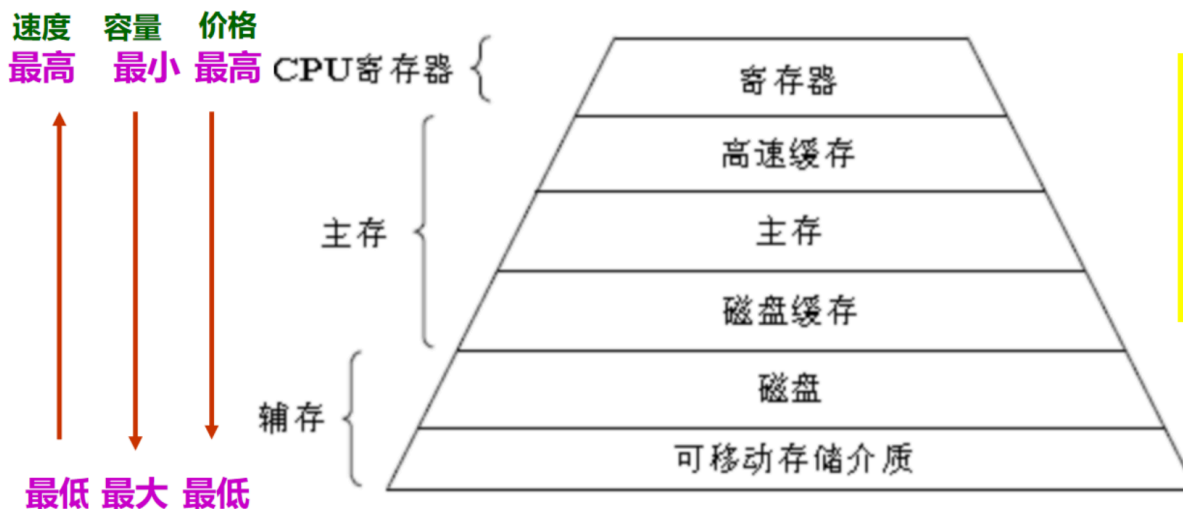
第四章 存储器管理

针对近两年题目，考虑一下这章会怎么考：

- 可能考页面置换的计算题
- 可能考逻辑地址转换的计算题
- 可能考有快表的情况下的访问时间
- 可能出现在前面问答，也可能出现在后面大题，理解逻辑过程、算法为主，把握好细节

4.1 存储器的结构层次

- 寄存器、高速缓存、主存储器和磁盘缓存均属于操作系统存储管理的管辖范畴，掉电后它们存储的信息不再存在。
- 磁盘和可移动存储介质属于设备管理的管辖范畴，它们存储的信息将被长期保存。



1.磁盘缓存：内存中一块存储区，对应于某固定磁盘，临时存储磁盘数据（如，数据预取）

2.存储器管理的目的和功能：

- 主存储器的分配和管理（记录存储区域状态、分配、释放）
- 提高主存储器的利用率（使多道程序能动态地共享主存，最好能共享主存中的信息）
- “扩充”主存容量（为用户提供比主存的存储空间还大的地址空间，虚拟存储器、自动覆盖技术）
- 存储保护（各道用户作业都在所分配的存储区内操作，互不干扰）

3.存储分配的三种方式：（把一个作业运行需要的全部或部分信息分配到主存中）

- 直接指定方式：编程序或者编译(汇编)时，使用实际存储地址。
- 静态分配方式：目的程序可从其地址空间的零地址开始，当装配程序对其进行连接装入时才确定它们在主存中的相应位置
- 动态分配方式：作业在存储空间中的位置也是在其装入时确定，能接受不可预测的分配和释放存储区域的请求，实现个别存储区域的分配和回收

4.基本概念：

- 逻辑地址（相对地址、虚地址）：首地址为0，其余指令中的地址都相对于首地址而编址
- 物理地址（绝对地址，实地址）：内存中存储单元的地址，可直接寻址
- 名空间：一个用高级语言编制的源程序
- 地址空间：逻辑（相对）地址的集合，“虚”
- 存储空间：主存中物理单元的集合，“实”

4.2 程序的装入和链接

1.将一个用户源程序变成一个可在内存中执行的程序经过3步骤：

- 编译

- 链接：形成一个完整的装入模块
- 装入：将装入模块**装入内存**

2.三种装入方式：

- 绝对装入方式：编译程序将产生实际存储地址（绝对地址）的目标代码，装入程序按照装入模块中的地址，将程序和数据装入内存
- 可重定位装入方式：经编译得到的目标模块中为相对地址（通常从0开始），装入模块中的逻辑地址与实际装入内存的物理地址不同，装入内存时，相对地址要修改以得到正确的物理地址（修改的过程称为重定位）
- 动态运行时装入方式：采用动态重定位方法将程序装入内存

重定位：

- 静态重定位：地址变换是在装入内存时一次完成的，且以后不能移动
- 动态重定位：把相对地址转换为绝对地址的过程推迟到程序执行时进行，最简单的办法是利用一个重定位寄存器(RR)，不根据CPU给出的有效地址去访问主存，而是将有效地址与重定位寄存器中的内容相加后得到的地址作为访问主存的地址。

3.三种链接方式：

- 静态链接：运行前链接，链接完不再拆开
- 装入时动态链接：边装入边链接，装入的过程中修改相对地址和外部引用地址
- 运行时动态链接：装入时不链接，运行时发现引用的地址是相对地址或者外部地址时，才发起链接

4.3 连续分配方式（重点）

为用户程序分配一个连续的内存空间

内零头：分配给用户但用户没有使用的空间

外零头：没有分配但无法分配的空间

1.连续分配有三种方式：

- **单一连续分配方式**：单用户系统在一段时间内，只有一个进程在内存。
内存分为两个区域，一个供操作系统使用，一个供用户使用。
- **分区式分配方式**：系统把内存用户区划分为若干分区，分区大小可以相等，也可以不等。一个进程占据一个分区。
可以分为：(1)固定分区 (2)动态分区（可变分区）
- **动态重定位分区分配**

2.固定分区式分配：

- 有内零头产生
 - 采用分区表记录分区的大小和使用情况
-

3.动态分区分配：

- 有数目固定、大小可变的

- 有数目、大小都可变的

基于顺序搜索

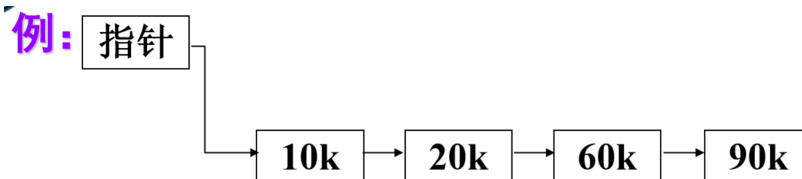
- (1)最佳适应算法(Best Fit)
- (2)最坏适应算法(Worst Fit)
- (3)首次适应算法(First Fit)
- (4)循环首次 (下次) 适应算法(Next Fit)

基于索引搜索

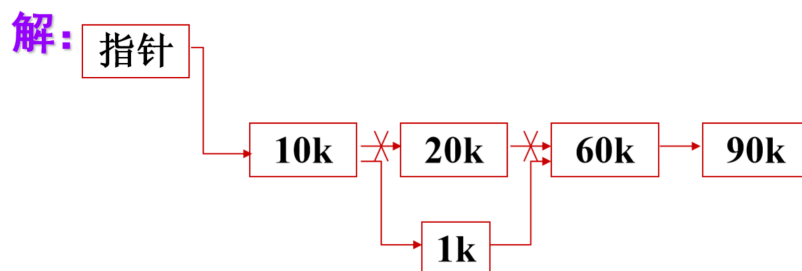
- (5)快速适应算法(Quick Fit)
- (6)伙伴系统
- (7)哈希算法

- 1.最佳适应算法(Best fit: BF)

- 将存储空间中所有的空白区按其大小递增的顺序链接起来
- 总是寻找大小最接近作业所要求的存储区域

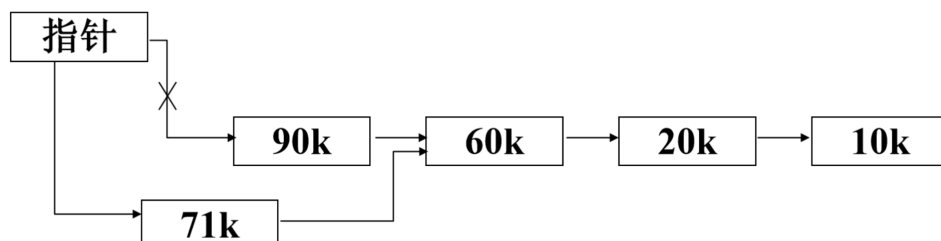


有四块空白区，来了一个作业需分配19k内存。



- 2.最坏适应算法(Worst fit: WF)

- 空闲分区按容量从大到小排成空闲分区表
- 总是挑选一个最大的空闲分区分割给作业使用



- 3.首次适应算法(First Fit: FF)

- 空闲分区链以地址递增的次序链接
- 从链首开始顺序查找，直至找到一个大小能满足要求的空闲分区为止

- 4.循环首次适应算法(下次适应算法) (Next fit: NF)

- 将所有的空闲分区构成一个循环链表

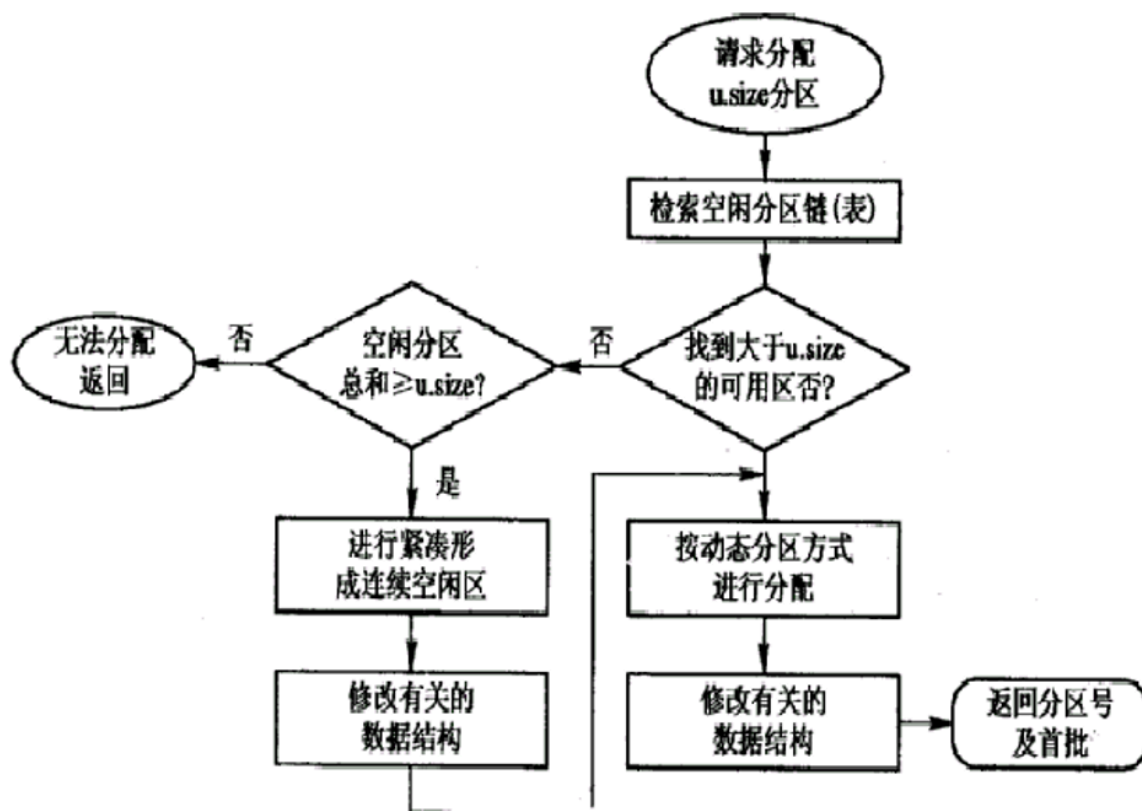
- 每次查找时不是从头开始，而是从上次结束位置开始
- 5.快速适应算法 (Quik fit:QF)
 - 对于每一类具有相同容量的所有空闲分区，单独设立一个空闲分区链表
 - 系统中存在多个空闲分区链表
 - 根据进程的长度，寻找到能容纳它的最小空闲分区链表，并取下第一块进行分配即可
- 6.伙伴系统
 - 固定分区问题：并发执行的进程数量受到限制，内部碎片影响内存利用率
 - 动态分区问题：算法复杂，回收分区时系统开销大
 - 伙伴系统是一种折中方案
 - 在伙伴系统中，可用内存块的大小为 2^k ($1 \leq k \leq m$)

➤ 进程请求大小为 n 的存储空间时，

- 首先计算一个 i 值，使 $2^{i-1} < n \leq 2^i$;
- 在空闲分区大小为 2^i 的空闲分区链表中查找。
 - if** 找到，即把该空闲分区分配给进程。
 - else** 在分区大小为 2^{i+1} 的空闲分区链表中寻找;
 - //表明长度为 2^i 的空闲分区已经耗尽
 - if** 找到大小为 2^{i+1} 的空闲分区
 - 把该空闲分区分为相等的两个分区（一对伙伴），其中一个用于分配，另一个加入分区大小为 2^i 的空闲分区链表中。
 - else** 查找大小为 2^{i+2} 的空闲分区.....

- 7.哈希算法

4.动态重定位分区分配



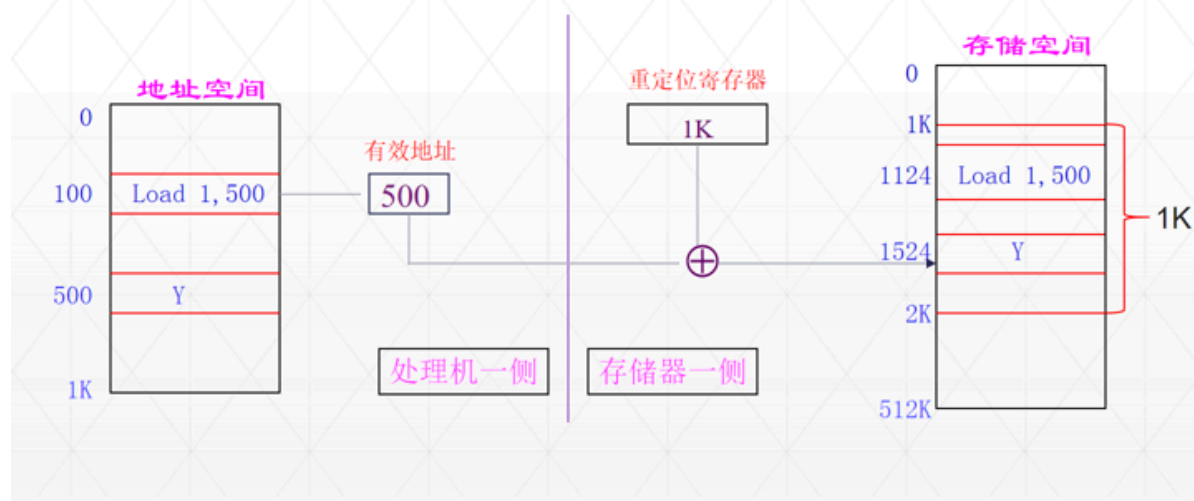
启动紧凑的时机：1.载入大程序，而无可用空间时，系统自动启动紧凑，2.发现碎片较多，系统比较空闲时。

二、动态重定位

□ 改变程序驻留位置后的地址重定位问题解决方法：

动态重定位机制需要硬件的支持，增加**重定位寄存器**（分区首址寄存器），保存分区起始地址，当进行紧凑内存空间时，只需移动分区内容，然后用新的分区起始地址重置分区首址寄存器。

程序中的相对地址 = 重定位寄存器 + 相对地址



4.4 对换

把内存中暂时不能运行的进程或者暂时不用的程序和数据，调出到外存上，以便腾出足够的内存空间再把已具备运行条件的进程或进程所需要的程序和数据，调入内存。

为了实现进程对换，系统必须能实现三方面的功能：

- 对换空间的管理
- 进程的换出
- 进程的换入

4.5 分页存储管理方式

离散分配：程序在内存中不一定连续存放

1.根据离散时的基本单位不同，可分为三种：

- 分页存储管理
- 分段存储管理
- 段页式存储管理

2.离散分配的优点：

- 没有外零头
- 仅有小于一个页面的内零头

3.离散分配基本思想：

- 将程序地址空间分页
- 将内存空间分块
- 内存一块可以装入程序一页

分页式存储管理

1.基本方法：

- 页面、页：将一个进程的逻辑地址空间分成若干个大小相等的片，称为页面或页
- 块、页框：把内存空间分成与页面相同大小的若干个存储块，称为（物理）块或页框
- 将进程中的若干个页分别装入到多个可以不相邻接的物理块中

2.页表：

- 为了能在内存中找到每个页面所对应的物理块
- 系统为每个进程建立了一张页面映像表，简称页表
- 每个进程对应 1 个页表
- 页表中包括页号、物理块号

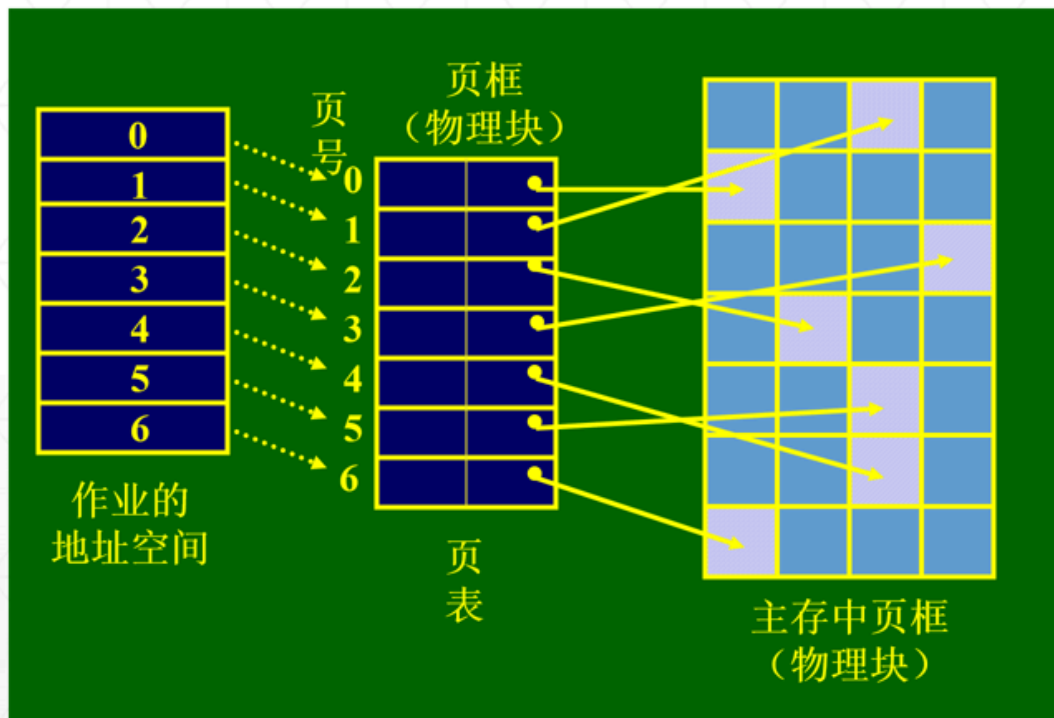
3.作业表：

- 整个系统1张，记录作业的页表情况

- 包含进程号、页表长度、页表始址等信息

4.空闲块表:

- 整个系统1张，记录主存当前空闲块

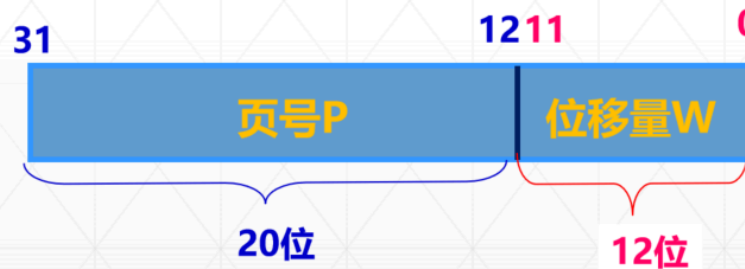


5.地址结构:

在分页存储管理方式中，任何一个逻辑地址都可转变为:

页号+页内位移量

◆ 如：有一个32位的逻辑地址，可转化为如下方式:



**0~11位表示页内位移量，则每页的大小为 $2^{12} = 4\text{KB}$ 。
12~31位表示页号， $2^{20} = 1\text{M}$ ，即最多允许有1M个页面。**

逻辑地址A、页面大小L:

页号
页内位移量

$$P = \text{INT}[A/L]$$
$$W = A \bmod L$$

例：设虚地址为 $(357101)_8$ 每一块为1K字节

块（页）的大小： $1K=2^{10}$

$$(357101)_8 = (011,101,111,001,000,001)_2$$

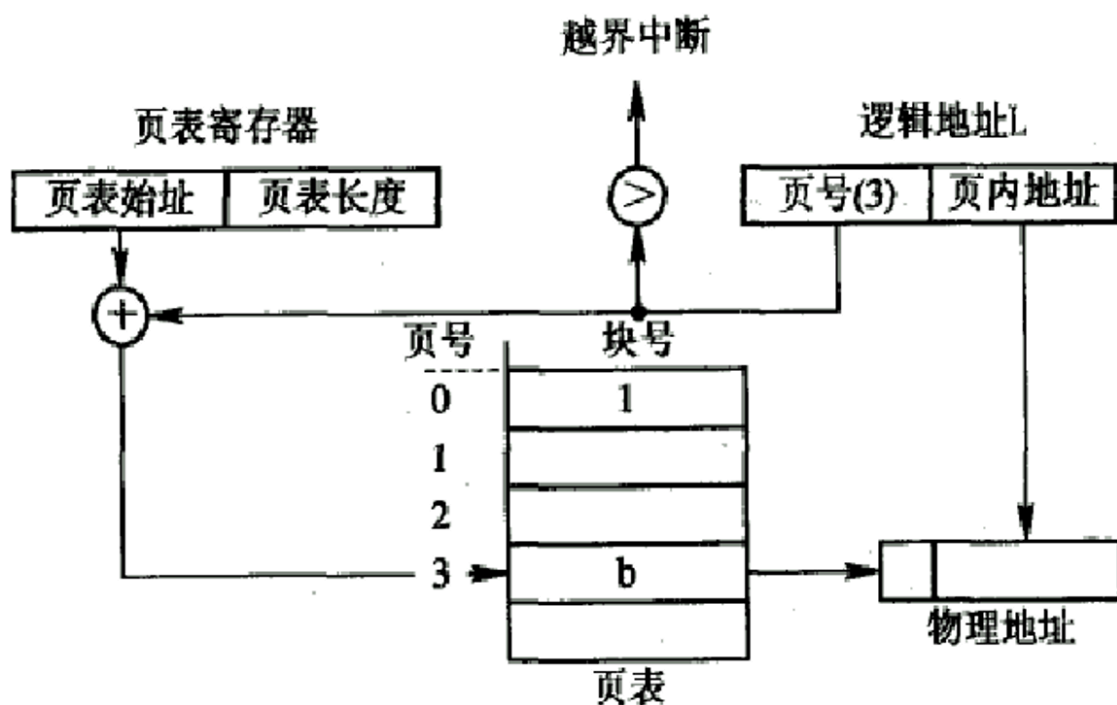
$$= (01,110,1111,001,000,001)_2$$

1 6 7 1 1 0 1

位移量为 $(1101)_8$, 页号为 $(167)_8$

块号由页表指定，位移量在变换过程中不变

6.地址变换：



7.快表

- 没有快表的时候，每次要访问两次内存：

第一次访问页表，以得到物理地址

第二次访问物理地址，以得到数据

- 所以，为进程页表设置一个专用的高速缓冲存储器，称为快表，或联想存储器
- 专门保存当前进程最近访问过的一组页表项
- 先查快表，命中，则计算出物理地址
- 没命中，则查页表，同时更新数据

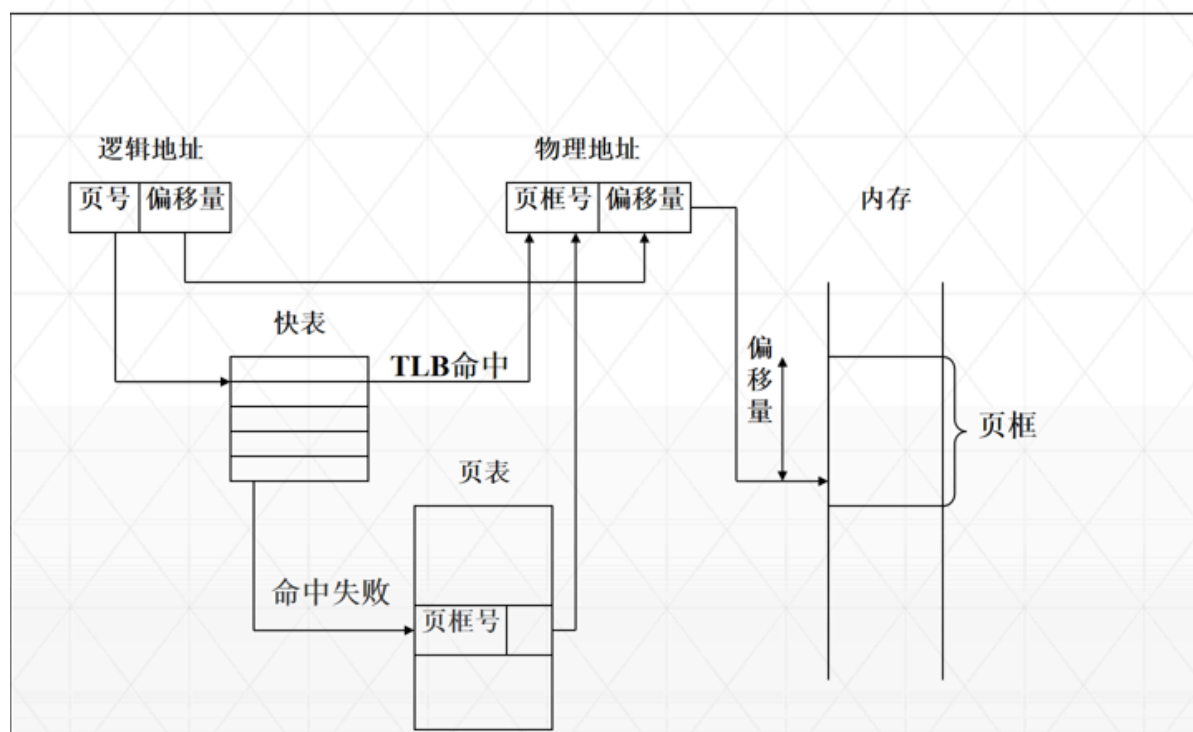


图 具有快表的分页系统地址变换过程

4.5.3 访问内存的有效时间 EAT

例：设访问主存时间为200ns,访问联想存储器为40ns,命中率为90%，则平均存取时间为多少？

解：

查页表两次访存： $200 + 200 + 40 = 440\text{ns}$

查快表，平均为： $40 + 200 = 240\text{ns}$

$(200 + 40) \times 90\% + (200 + 200 + 40) \times 10\% = 260\text{ns}$

8.两级页表

把页表本身按固定大小分成为一个个页面

两级页表解决了大页表无需大片连续存储空间问题

但并未减少用较少内存去存放大页表问题，有关此类问题的成功解决方案放在虚拟存储器管理中。

9.多级页表 (P125)

10.反置页表 (IPT)

出发点：逻辑空间越来越大，页表占内存也越来越大，为了解决大页表问题占内存多现象，减少内存开销，避免一个进程一个页表。

思想：为主存中的每一个物理块建立一个页表项并按照块号排序



反置页表

内存块号

0		
1		进程k1 1页
2		进程k2 1页
3		进程k1 3页
4		进程k1 4页
5		
6		进程k1 2页
7		进程k2 2页
8		
9		进程k2 3页
	⋮	
n		

常采用部分装入，所以必须为每个进程建立一个外部页表。当该页不在主存时，需要访问外部页表。

反置页表只索引正在运行的进程页面

因为IPT都大，所以利用进程标识符和页号去检索是相当费时的。采用Hash表来检索

电子科技大学

11.对分页存储管理整体的评价：

- 彻底消除了外零头,仅存在很少的内零头
- 一个页面不能实现某种逻辑功能，用户看到的逻辑地址是一维的
- 不易于实现存储共享，也不便于程序的动态链接
- 页面的划分并没有逻辑意义

4.6 分段式存储管理

具有逻辑上的意义

1.分段原因：

- 方便编程
- 信息共享
- 动态链接
-

2.基本原理：

- 作业地址空间按逻辑信息的完整性被划分为若干个段
- 每段有段名（或段号），每段从0开始编址
- 对所有地址空间的访问均要求两个成分：(1)段的名称； (2)段内地址。

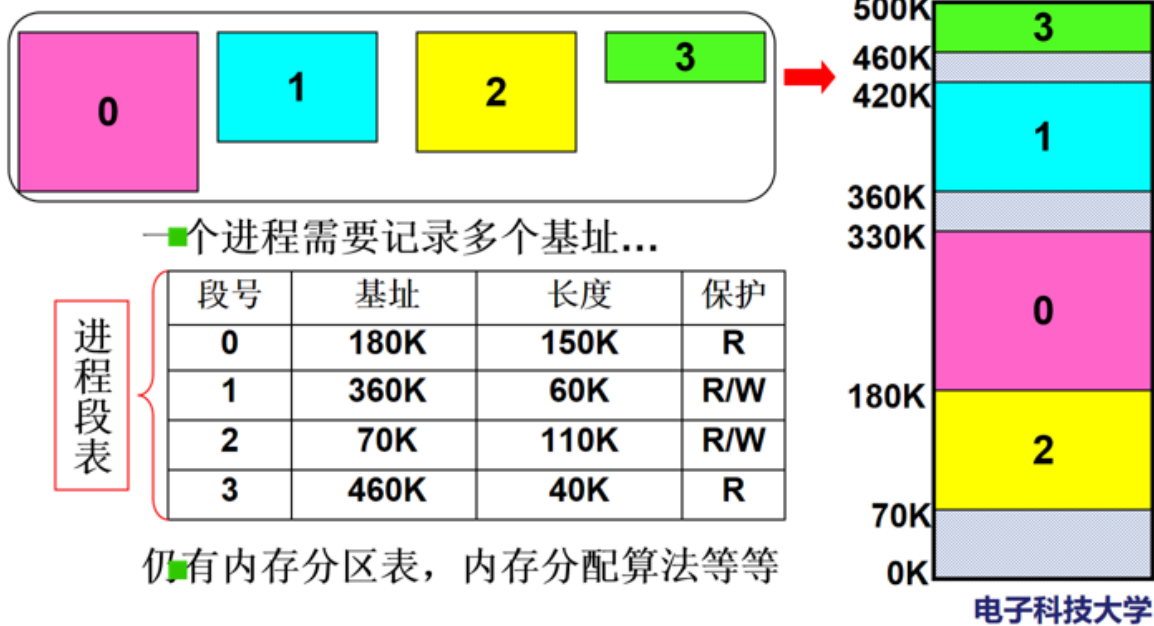
3.段表：

- 为每个分段分配一个连续的分区
- 而进程中的各个段可以离散地移入内存中不同的分区中
- 每个进程建立一张段映射表，简称“段表”
- 记录了该段在内存中的起始地址、段的长度



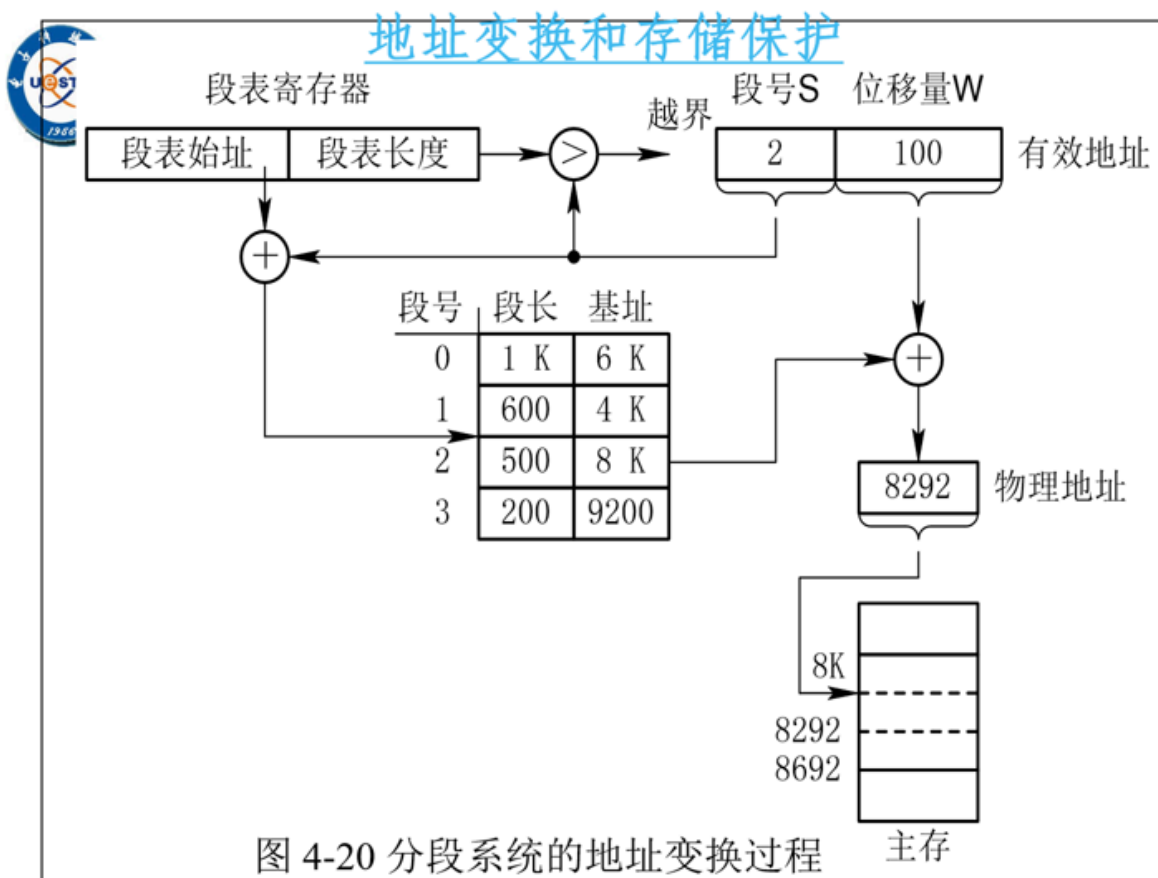
将段放入内存 引入段表

■ 分段制造了二维空间，而内存是一维的



4.段表寄存器：

- 实现快速地址变换
- 用来存放当前执行进程的段表在物理内存中的起始地址



5.对分段的评价：

- 有效消除了内零头
- 允许子程序独立编译和修改
- 容易实现存储共享
- 很容易满足程序段的动态增长需要

6.分页和分段比较:

- 页是信息的物理单位, 段则是信息的逻辑单位
- 页的大小固定且由系统决定, 段的长度却不固定, 决定于用户所编写的程序
- 分页的作业地址空间是一维的, 分段的作业地址空间则是二维的

7.信息共享:

- 分页系统实现程序段的共享较为困难
- 分段易于实现段的共享和段的保护

例如: 一个多用户系统可接纳40个用户, 它们都执行一个文本编辑程序(ED), ED代码共160K, 每个用户还有40K的数据区(DA)

- 分页共享、分段共享

8.段页式存储管理方式: 分段和分页相结合

- 先将用户程序分段, 每段内再划分成若干页
- 每段有段名(段号), 每段内部的页有一连续的页号
- 地址空间由**段号S**、**段内页号P** 和**页内相对地址W**构成



地址变换

➤ 设置段表、段内页表

段号	段长	页表始址	页表长

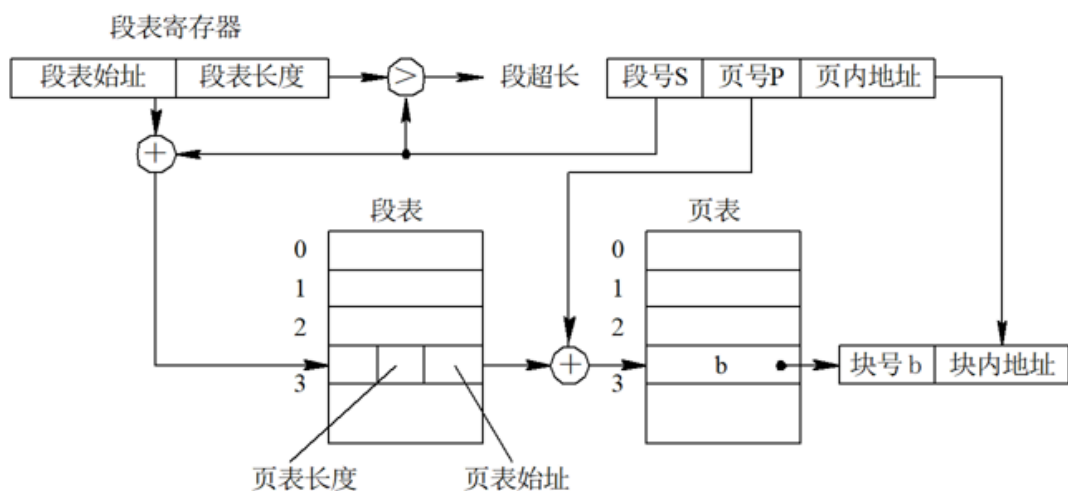
页号	块号
0	2
1	1
2	40

页号	块号
0	10
1	9
2	20

电子科技大学



段页式系统的地址变换机构



电子科技大学

在段页式存储管理方式中，每访问一次数据，需访问三次内存：

第一次访问内存中的段表

第二次访问内存中的页表

第三次访问相应数据

9.对段页式存储管理方式的评价：

- 综合了分段和分页技术的优点
- 系统的复杂度和管理开销也大大增加
- 适合于大、中型计算机系统，不太适合小型、微型计算机系统

4.7 虚拟存储器

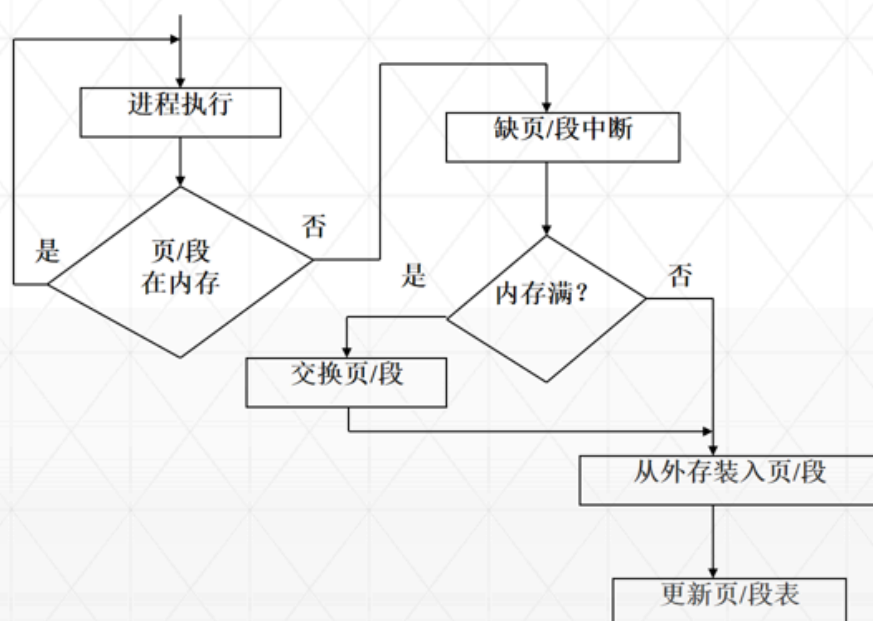
为解决大作业无法全部装入内存、内存不足以容纳大量作业的问题，从逻辑上增加内存容量

常规存储器：一次性（一个作业全部装入）、驻留性（一直驻留到作业完成）

1.基本工作情况：

- 作业运行前，仅将那些当前要运行的页面（段）装入内存启动运行，其余暂在外存
- 进程运行时，如果它所需要访问的页面（段）已经装入内存，则可以继续执行下去
- 如果其所需要访问的页面（段）尚未装入内存，则发生缺页（段）中断，进程阻塞
- 系统将启动请求调页（段）功能将其调入内存。
- 若此时内存满，则利用置换功能，将内存中暂时不用的部分页面（段）调至外存，再将所需页面（段）调入。
- 再将进程所需的页（段）装入内存，唤醒阻塞的进程，使之重新参与调度执行。

实现虚拟存储的典型过程



2.逻辑容量：由内存容量和外存容量之和所决定

3.请求分页系统：

- 在纯分页系统的基础上增加了请求调页、页面置换两大功能
- 所形成的页式虚拟存储系统

4.请求分段系统：

- 在纯分段系统的基础上增加了请求调段、分段置换两大功能
- 所形成的段式虚拟存储系统

5.段页式虚拟系统：

- 建立在段页式系统的基础上的，通过增加了请求调页、页面置换两大功能
- 所形成的段页式虚拟存储系统

6.虚拟存储器的特征：

- 多次性：一个作业被分成多次调入内存运行
- 对换性：作业的运行过程中进行换进、换出

(虚拟性是以多次性和对换性为基础的；而多次性和对换性又必须建立在离散分配的基础上。)

- 虚拟性：能够从逻辑上扩充内存容量，使用户所看到的内存容量远大于实际内存容量

7.抖动

4.8 请求分页存储管理方式

整体逻辑

1.工作原理：

- 只将作业的一部分装入内存，其余的放在辅存
- 一旦发现访问的页不在主存中，则发出缺页中断，由OS将其从辅存调入主存
- 如果内存无空块，则根据某种算法选择一个页淘汰以便装入新的页面

2.为了实现**请求调页**、**页面置换**两大功能，系统必须提供如下的硬件支持：

- 请求分页的页表机制
- 缺页中断机构
- 地址变换机构

3.页表机制

请求分页系统中的每个页表项如下：

页号	页框号Q	状态位D	访问位A	修改位M	外存地址
----	------	------	------	------	------

状态位：是否调入内存

访问位：记录本页在一段时间内被访问的次数，或最近已有多长时间未被访问

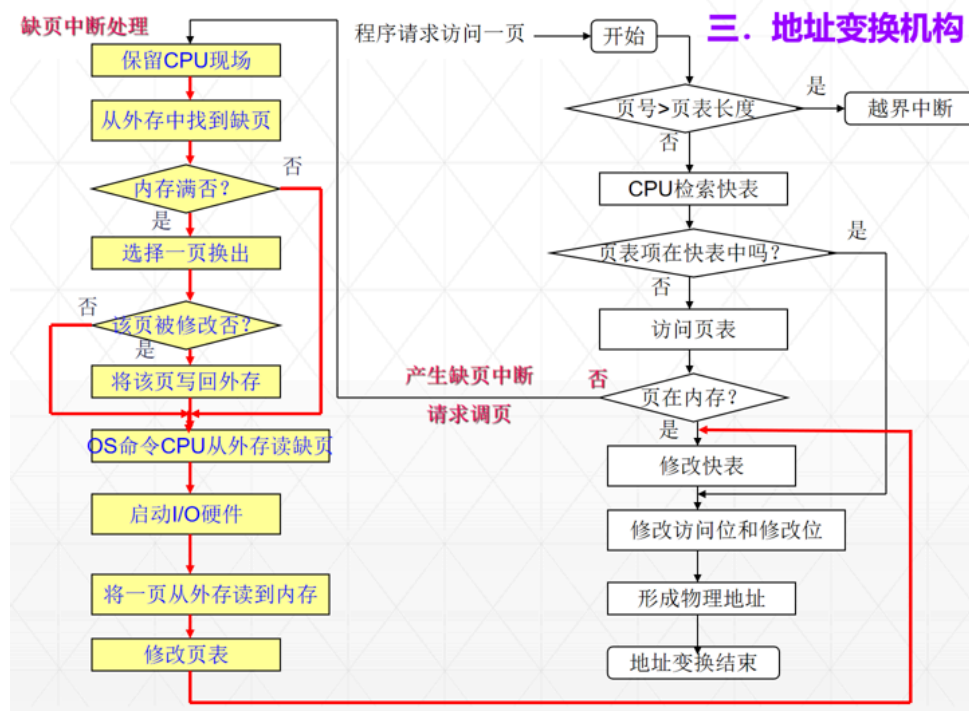
修改位：该页在调入内存后是否被修改过

4.缺页中断机构

缺页中断是一种特殊的中断：

- 在指令执行期间产生和处理中断信号
- 一条指令可能引起多次不同的页面故障

5.地址变换机构



内存分配策略和分配算法

1.最小物理块数：是指能保证进程正常运行所需的最少物理块数

2.物理块的分配策略：

- 固定分配局部置换：为每个进程分配一定数目的物理块，在整个运行期间都不再改变
- 可变分配全局置换：
 - 先为系统中的每个进程分配一定数目的物理块，而OS自身也保持一个空闲物理块队列
 - 当空闲物理块队列中的物理块用完时，OS才能从内存中选择一页调出
 - 该页可能是系统中任一进程的页
- 可变分配局部置换：
 - 为每个进程分配一定数目的物理块
 - 只允许从该进程在内存的页面中选出一页换出
 - 在进程运行过程中统计进程的缺页率，如果缺页率高，则为其增加一定的内存页，否则适当减少

3.物理块的分配算法

- 平均分配算法

- 按比例分配算法（根据进程大小）
- 考虑优先权的分配算法（根据优先权适当增加份额）

调页策略

1.预调页：

- 当进程创建时，预先为进程装入多个页面
- 缺页中断时，系统为进程装入指定的页面以及与之相临的多个页面

2.请求调页：

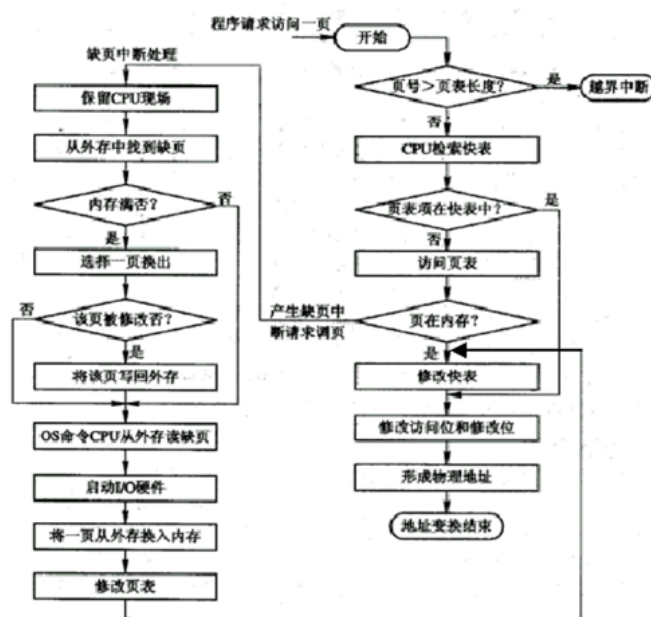
- 仅当进程执行过程中，通过检查页表发现相应页面不在内存时，才装入该页面

3.从何处调页，三种情况：（简单看看）

- 系统拥有足够的对换区空间
 - 进程运行前将全部有关文件从文件区拷贝到对换区。
 - 这时可以全部从对换区调入所需页面，以提高调页的速度。
- 系统缺少足够的对换区空间
 - 这时凡是不会被修改的文件，都直接从文件区调入；
 - 而当换出这些页面时，由于它们未被修改而不必再将它们换出（直接丢弃），以后再调入时，仍从文件区直接调入。
 - 但对于那些可能被修改的部分，在将它们换出时，便须调到对换区，以后需要时，再从对换区调入。
- UNIX方式
 - 由于与进程有关的文件都放在文件区，应从文件区调入。
 - 凡是未运行过的页面，都应从文件区调入。
 - 而对于曾经运行过但又被换出的页面，由于是被放在对换区，因此在下次调入时，应从对换区调入。
 - 允许页面共享

4.页面调入过程

3. 页面调入过程?



页面置换算法

1. 最佳置换算法(Optimal) (OPT)

- 从主存中移出永远不再需要的页面;
- 如无这样的页面存在, 则应选择最长时间不需要访问的页面。

2. 先进先出置换算法(FIFO)

- 总是选择作业中驻留时间最长(即最老)的一页淘汰

3. 最近最久未使用(LRU)置换算法

- 选择在最近一段时间内最久不用的页面予以淘汰

对LRU算法的支持:

1. 移位寄存器。用于记录某进程在内存中各页使用情况

- 当进程访问某物理块时, 要将相应寄存器的最高位 R_{n-1} 位置成1。
- 系统每隔一定时间 (例如100 ms) 将寄存器右移一位。
- 具有最小数值的寄存器所对应的页面, 就是最近最久未使用的页面

2. 栈。用于保存当前进程使用的各个页面的页面号

- 每当进程访问时某页面时, 便将该页面号从栈中移出, 压入栈顶。
- 这样栈底则是最近最久未使用页面的页面号

4. 最少使用置换算法LFU

- 记录每个页面的访问次数, 最少访问的页面首先考虑淘汰

为页面设置移位寄存器。

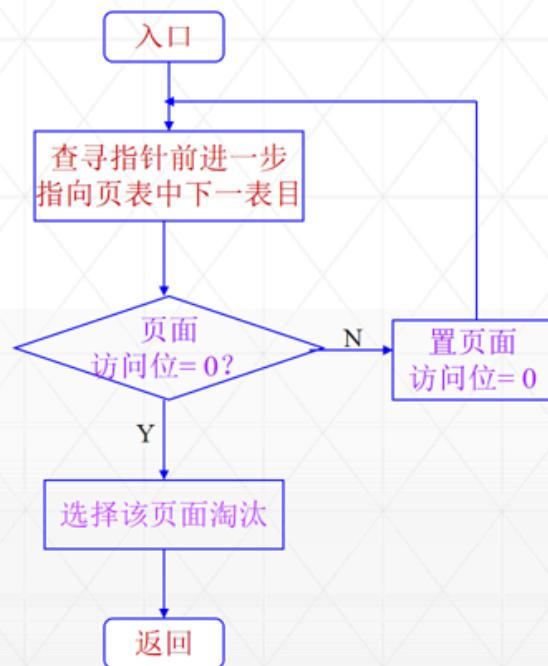
与LRU的区别:

$R1=10000000$; $R2=01110100$

5.简单的Clock置换算法 (NRU)

- 为每页设置一位访问位，再将内存中的所有页面都通过链接指针链接成一个循环队列
- 当某页被访问时，其访问位被置1
- 置换程序从上次停止位置的下一位开始，检查页面的访问位
- 如果是0，就选择该页换出；
- 若为1，则重新将它置0，暂不换出

流程图



示例图

块号	页号	访问位	指针
0			
1			
2	4	0	
3			
4	2	1	
5			
6	5	0	
7	1	1	

替换指针



采用Clock置换算法的缺页中断与页面置换过程

页面引用序列

2

3

2

1

5

2

4

5

3

2

5

2

2*	2*	2*	2*	5*	5*	5*	5*	3*	3*	3*	3*
	3*	3*	3*	3	2*	2*	2*	2	2*	2*	2*
			1*	1	1	4*	4*	4	4	5*	5*

F

F

F

-

-

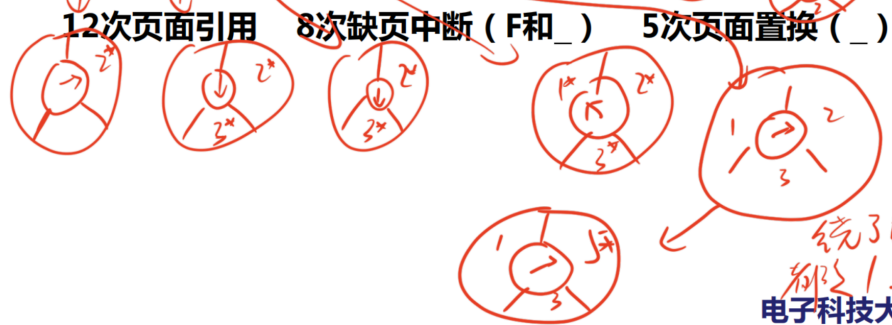
-

-

-

-

-



6.改进型Clock置换算法 (简单看看)

▪ 执行过程可分成以下三步:

(1) 从指针所指示的当前位置开始, 扫描循环队列, 寻找 $A=0$ 且 $M=0$ 的第一类页面, 将所遇到的第一个页面作为所选中的淘汰页。在第一次扫描期间不改变访问位 A 。

(2) 如果第一步失败, 即查找一周后未遇到第一类页面, 则开始第二轮扫描, 寻找 $A=0$ 且 $M=1$ 的第二类页面, 将所遇到的第一个这类页面作为淘汰页。在第二轮扫描期间, 将所有扫描过的页面的访问位 A 都置0。

(3) 如果第二步也失败, 亦即未找到第二类页面, 则将指针返回到开始的位置, 并将所有的访问位复0。然后重复第一步, 如果仍失败, 必要时再重复第二步, 此时就一定能找到被淘汰的页。

4.9 缺页率对有效访问时间的影响



4.8 缺页率对有效访问时间的影响

- 有效访问时间是指访问存储器所需时间的平均值。
- 假设使用了快表，则CPU访问内存时有以下三种情况：

设内存读写周期为 t ，查找快表时间为 λ ，缺页中断处理时间为 ε

缺页中断处理时间由三部分组成：

- 缺页中断服务时间
- 页面传送时间：包括读缺页和写置换页的时间
- 进程重新执行时间

由于缺页中断服务时间及进程重新执行时间较短，这里仅考虑页面传送时间。

电子科技大学



- 页面在内存且页表项在快表中：只需一次访问内存

$$EAT = \lambda + t$$

- 页面在内存但页表项不在快表中：需两次访问内存，一次读取页表，一次读取数据，另外还需更新快表。

$$EAT = \lambda + t + t + \lambda = 2(\lambda + t)$$

- 页面不在内存：考虑查找快表时间、查找页表时间、缺页中断处理时间、更新快表时间、访问实际物理地址时间

$$EAT = \lambda + t + \varepsilon + \lambda + t = \varepsilon + 2(\lambda + t)$$

- 引入快表命中率为 α ，缺页中断率为 f ，则有效访问内存时间为：

$$EAT = \lambda + \alpha t + (1 - \alpha)[t + f(t + \varepsilon + \lambda) + (1 - f)(t + \lambda)]$$

最后这个公式的理解：

- 首先，肯定是先访问快表， λ
- 然后， α 的概率访问到，直接加 t

- $(1-\alpha)$ 的概率访问不到，再分缺页和不缺页讨论
-以此类推

抖动

工作集

4.10 请求分段存储管理方式

(大概看看就好)

1.工作原理:

- 程序运行之前，只需先调入若干个分段（不必调入所有的分段）
- 当所访问的段不在内存中时，可请求OS将所缺的段调入内存

2.段表机制:

段名	段长	段的基址	存取方式	状态位P	访问位A	修改位M	增补位	外存地址
----	----	------	------	------	------	------	-----	------

增补位：说明该分段是否允许扩展，此外如该段已被增补，则在写回辅存时，需另选择辅存空间；



2. 缺段中断机构

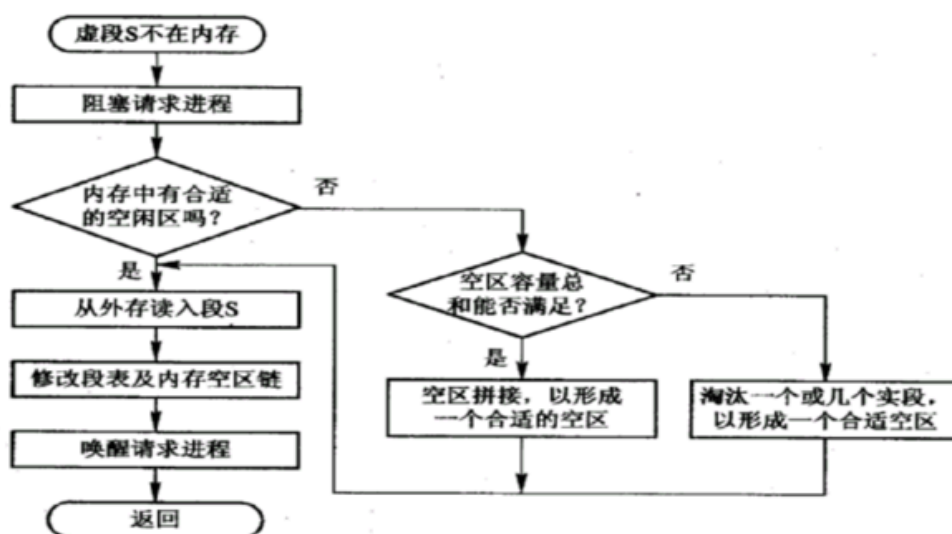
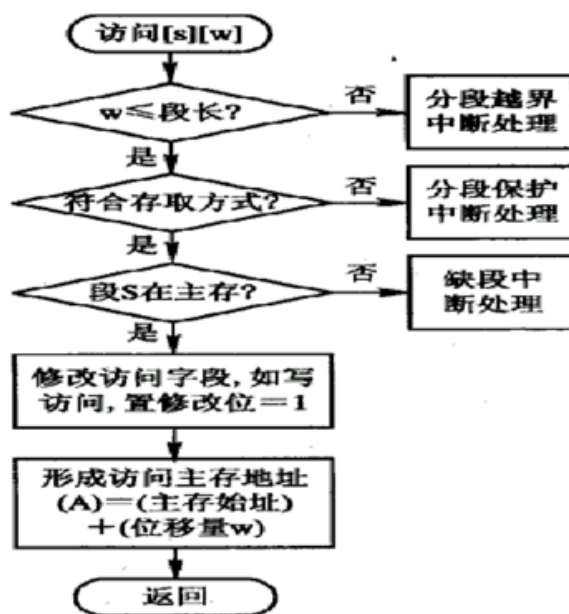


图 4 - 31 请求分段系统中的中断处理过程



3. 地址变换机构



电子科技大学

3.分段的共享与保护

- 共享段表.....
- 共享段的分配.....
- 共享段的回收.....
- 分段保护.....

第五章 输入输出系统

考虑：主要就是

- 后面的寻道算法

别的应该不重要

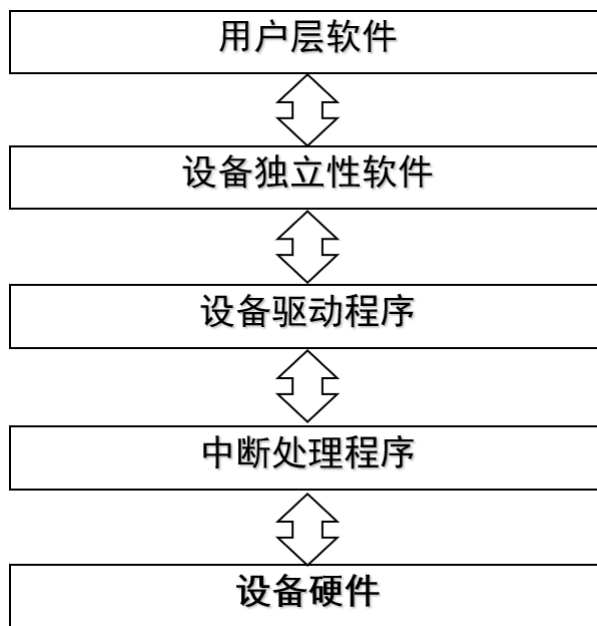
5.1 I/O系统简介

设备管理的对象：主要是I/O设备。

设备管理的基本任务：完成用户提出的I/O请求，提高I/O速率以及改善I/O设备的利用率。

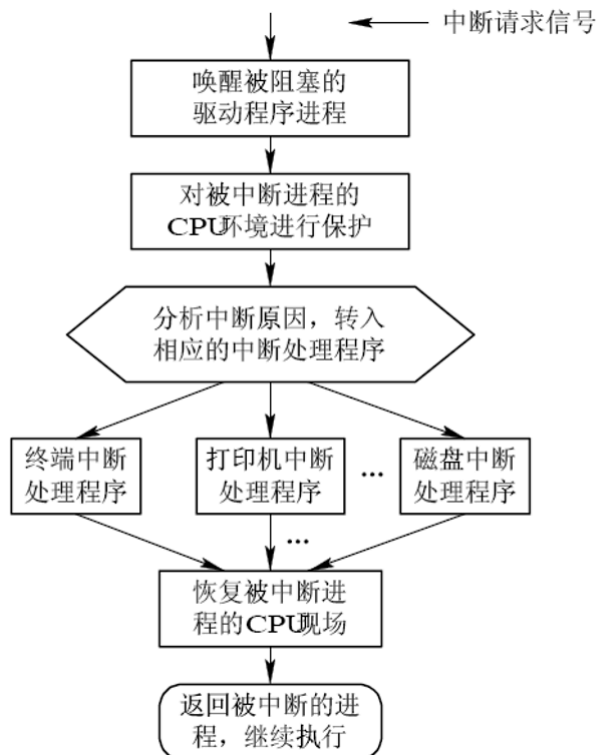
设备管理的主要功能有：缓冲区管理、设备分配、设备处理、虚拟设备及实现设备独立性等。

I/O系统的层次结构：



5.2 中断处理程序和设备驱动程序

中断处理程序



设备驱动程序

主要任务是启动指定设备。

- 驱动程序
- 中断处理程序

1. 设备驱动程序功能：

- 接收由I/O进程发来的命令和参数，并将命令中的抽象要求转换为具体要求
- 检查用户I/O请求的合法性，了解I/O设备的状态，传递有关参数，设置设备的工作方式
- 发出I/O命令

- 及时响应由控制器或通道发来的中断请求，并根据其中断类型调用相应的中断处理程序进行处理

2.对I/O设备的控制方式

- 使用轮询的可编程I/O方式
- 使用中断的可编程I/O方式
- 直接存储器访问方式
- I/O通道控制方式

5.3 与设备无关的I/O软件

在应用程序中，使用逻辑设备名称来请求使用某类设备；

而系统在实际执行时，还必须使用物理设备名称

因此，系统须具有将逻辑设备名称转换为某物理设备名称的功能

Spooling技术

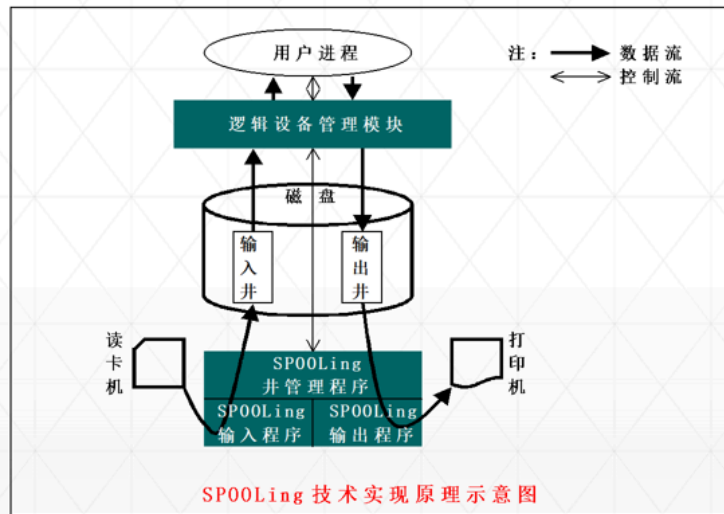
1.核心思想：

- 在快速辅助存储设备中建立I/O缓冲区
- 用于缓存从慢速输入设备流入内存的数据
- 或缓存从内存流向慢速输出设备的数据

2.通过SPOOLing技术便可将一台物理I / O设备虚拟为多台逻辑I / O设备，同样允许多个用户共享一台物理I / O设备

3.SPOOLing技术在输入输出之间增加了“输入井”和“输出井”的排队转储环节，以消除用户的“联机”等待时间

4.“输入井”和“输出井”则是在高速辅存（外存）中开辟的两个固定的转储区



5.4 磁盘系统及磁盘调度

前置

1. 提高磁盘 I/O 速度的主要途径：

- (1) 选择性能好的磁盘
- (2) 采用好的磁盘调度算法
- (3) 设置磁盘高速缓存 (Disk Cache)
- (4) 采用高度可靠、快速的容量磁盘系统--磁盘冗余阵列

2. 从磁盘读数据时：

访问时间 = 寻道时间 + 旋转延迟 + 传输时间

3. 寻道时间： T_s

$$T_s = m \times n + s$$

s: 启动磁臂的时间

n: 磁头移动 n 条磁道

m: 移动每一条磁道所花费的时间

4. 旋转延迟时间 T_r

$T_r = 1/(2r)$ (因为平均情况下, 需要旋 转半圈)

软盘旋转速度为 300 r/min或600 r/min, 这样, 平均 T_r 为50 ~ 100 ms

5.传输时间 T_t

把数据从磁盘读出或向磁盘写入数据所经历的时间

$$T_t = \frac{b}{rN}$$

r 为磁盘每秒钟的转数; N 为一条磁道上的字节数

磁盘调度

磁盘调度算法:

1.先来先服务FCFS

2.最短寻道时间优先SSTF (Shortest Seek Time First)

3.扫描算法 (SCAN) (电梯调度算法)

- 例如, 磁头自里向外移动, 并同时自里向外地访问
- 直至再无更外的磁道需要访问时, 才将磁臂换向自外向里移动

4.循环扫描算法 (CSCAN)

- CSCAN算法规定磁头单向移动
- 例如, 只是自里向外移动
- 当磁头移到最外的磁道并访问后, 磁头立即返回到最里的欲访问磁道

5.N步扫描算法N-Scan

- 将磁盘请求队列分成若干个长度为 N 的子队列
- 按照FCFS算法依次处理这些子队列
- 处理每一个子队列时, 则按照SCAN算法

6.FSCAN算法

- 实质上是 N 步SCAN算法的简化
- 将磁盘请求队列分成两个子队列
- 一个是由当前所有请求磁盘I/O的进程形成的队列, 由磁盘调度按SCAN算法进行处理
- 在扫描期间, 将新出现的所有请求磁盘I/O的进程, 放入另一个等待处理的请求队列

第六章 文件及文件系统

考虑，主要就是：

- 会考一个简答题：访问某个目录的过程、文件系统类型等等.....
- 会考一个混合索引的计算题（多级索引）
- （大概就这样）

6.1 文件系统概述

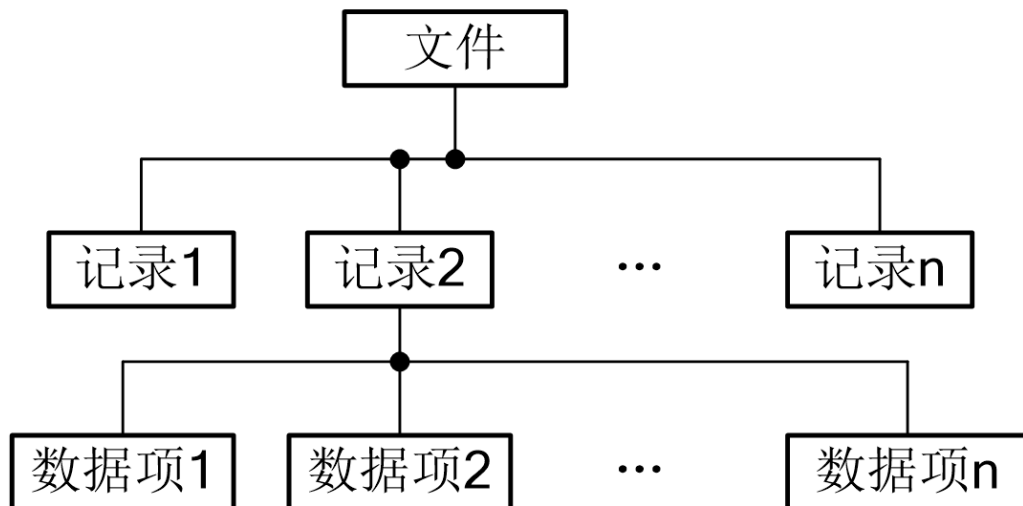
1.文件系统的功能：

- 有效地管理文件的存储空间；
- 管理文件目录；
- 完成文件的读/写操作；
- 实现文件共享与保护；
- 为用户提供交互式命令接口和程序调用接口。

2.文件系统的定义：

操作系统中的各类文件、管理文件的软件，以及管理文件所涉及到的数据结构等信息的集合。

3.文件、记录、数据项



数据项 :是最低级的数据组织形式，可把它分成以下两种类型：基本数据项、组合数据项

记录：一组相关数据项的集合，用于描述一个对象在某方面的属性

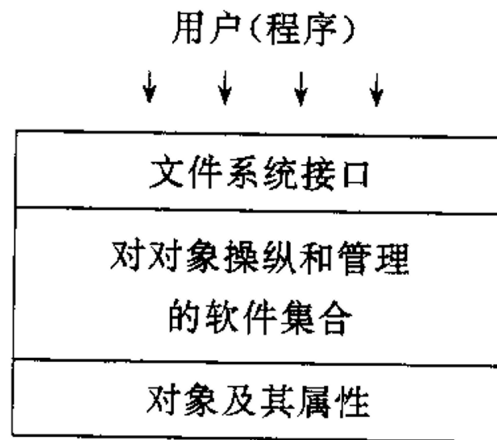
文件：由创建者所定义的、具有文件名的一组相关元素的集合。

- 在有结构的文件中，文件由若干个相关记录组成；
- 而无结构文件则被看成是一个字符
- 文件在文件系统中是一个最大的数据单位，它描述了一个对象集

4.文件分类：

- 按用途分类：系统文件、用户文件、库文件
- 按文件中数据的形式分类：源文件、目标文件、可执行文件
- 按存取控制属性分类：只执行文件、只读文件、读写文件
- 按组织形式和处理方式分类：普通文件、目录文件、特殊文件

5.文件系统模型：



1)对象及其属性：

- 文件管理系统管理的对象有：文件、目录、磁盘存储空间

2)对对象操纵和管理的软件集合：

- 这是文件管理的核心部分。
- 文件系统的功能大多是在这一层实现的

3)文件系统的接口

- 为方便用户使用文件系统，文件系统通常向用户提供两种类型的接口
- 命令接口、程序接口

6.2 文件的物理结构

文件的逻辑结构：从用户观点出发所观察到的文件组织形式

文件的物理结构：指文件在外存上的存储组织形式

从逻辑组织的角度看，文件由若干记录构成；

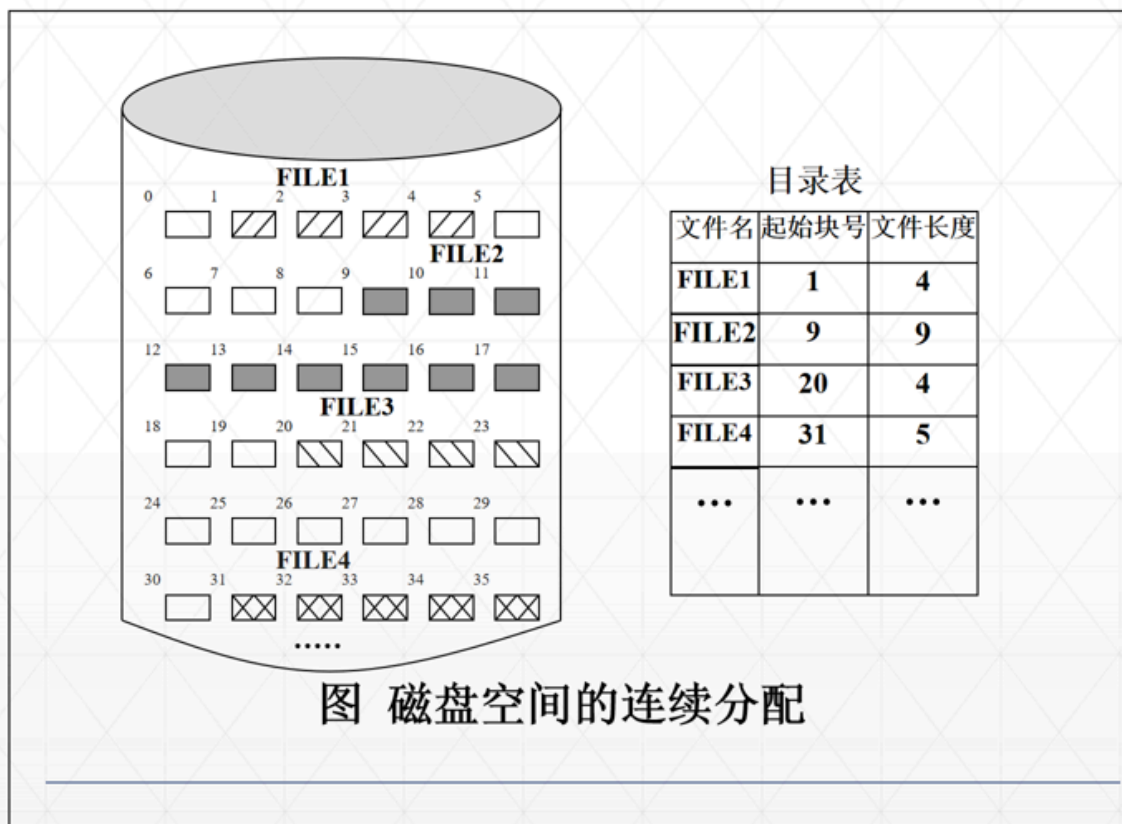
从物理组织的角度看，文件由若干数据块组成。

为文件分配外存空间时，常用的外存分配方法有：

- 连续分配
- 链接分配
- 索引分配

1.连续分配:

- 为每一个文件分配一组相邻接的盘块
- 文件目录中为每个文件建立一个表项, 其中记载文件的第一个数据块地址及文件长度

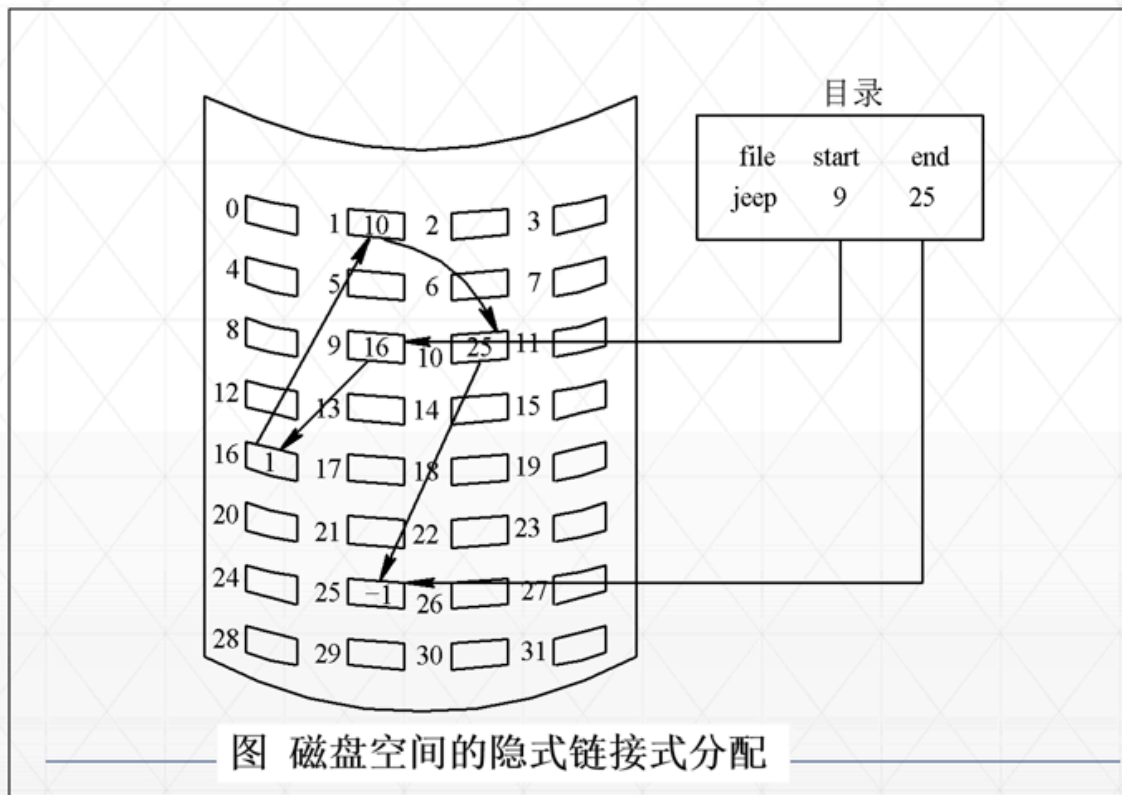


2.链接分配:

- 通过在每个盘块上的链接指针, 将同属于一个文件的多个离散的盘块链接成一个链表

1) 隐式链接:

- 在文件目录的每个目录项中, 都须含有指向链接文件第一个盘块和最后一个盘块的指针
- 每个盘块中都含有一个指向下一个盘块的指针



主要问题：它只适合于顺序访问，它对随机访问是极其低效的

2) 显式链接：

- 把用于链接文件各物理块的指针，显式地存放在**内存**的一张链接表中
- 整个磁盘仅设置一张文件分配表（FAT）
- 文件的第一个盘块号，作为文件地址被填入相应文件的FCB的“物理地址”字段中
- 查找记录的过程是在内存中进行的，因而不仅显著地提高了检索速度，而且大大减少了访问磁盘的次数
- 分配给文件的所有盘块号都放在该表中
- 该表称为文件分配表FAT (File Allocation Table)

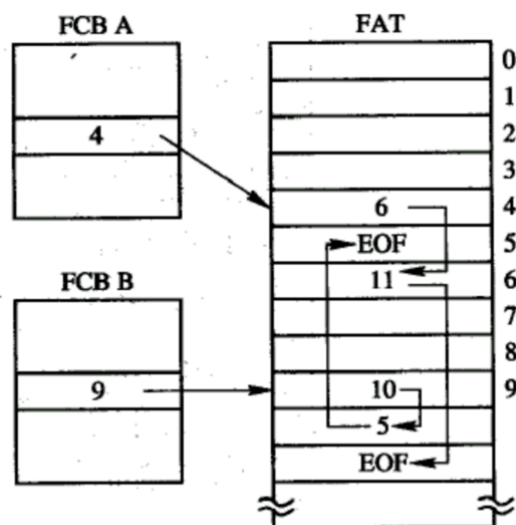


图 6 - 10 MS - DOS 的文件物理结构

3.索引分配:

出发点: 链接分配的问题:

- 不能支持高效的直接存取
- FAT需占用较大的内存空间, 只有将整个FAT 调入内存, 才能保证在FAT 中找到一个文件的所有盘块号

1) 单级索引分配:

- 为每个文件分配一个索引块(表)
- 再把分配给该文件的所有盘块号都记录在该索引块中
- 建立一个文件时, 只需在为之建立的目录项中填上指向该索引块的指针

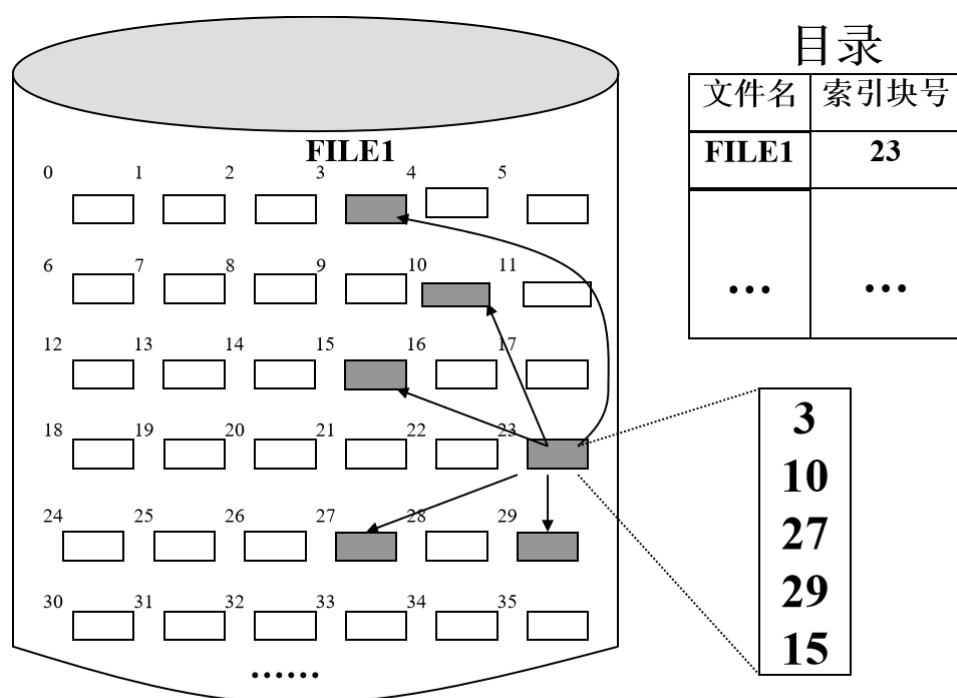


图 基于数据块分区的索引分配

图中: 第一个扇区, 23号, 存的不是有用的数据, 而是目录;

启动至少两次磁盘, 第一次读目录, 第二次读相应的数据

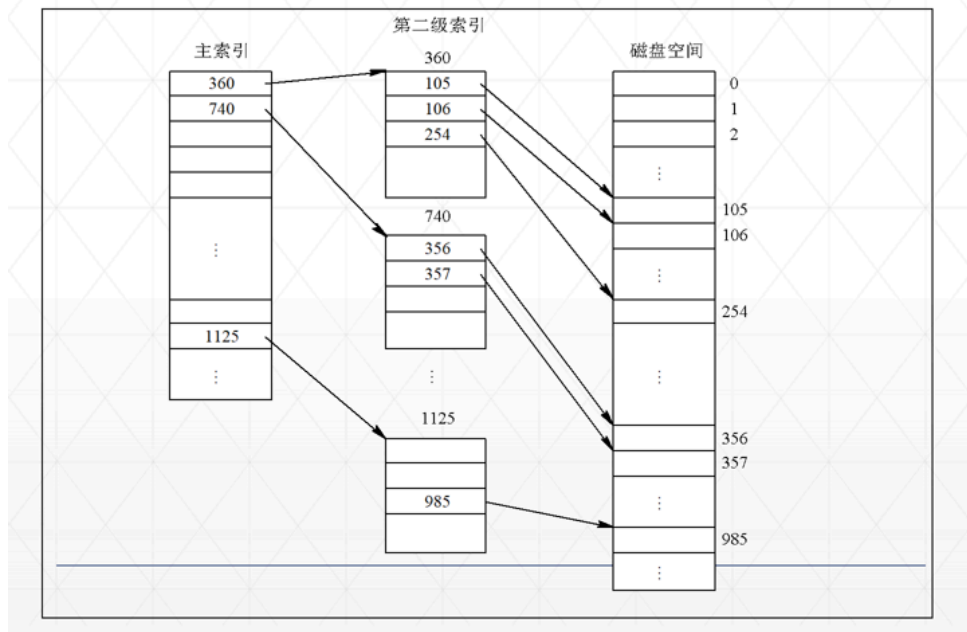
缺点:

- 大文件索引项较多, 可能使一个数据块容纳不了一个文件的所有分区的索引
- 索引块可能要花费较多的外存空间

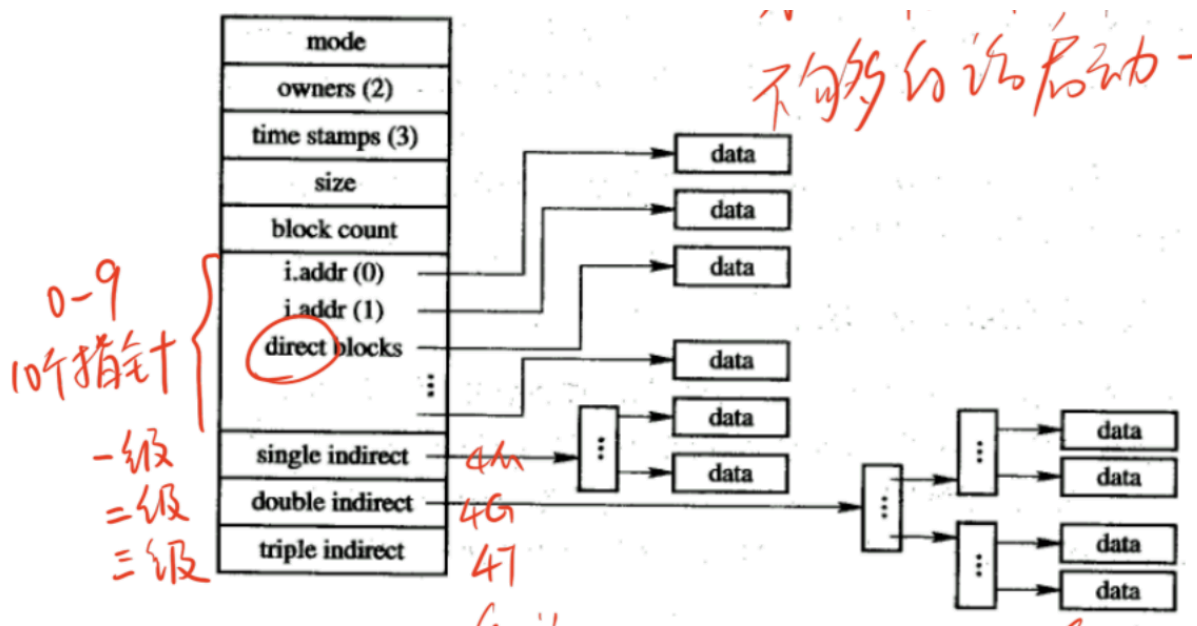
由单级的缺点, 引出:

2) 两级索引分配

两级索引分配



3) 混合索引方式



小文件，前10个扇区基本够用（这时，两次启动磁盘即可读出数据）

不够的话，再启动一次间接的.....

6.3 文件存储空间的管理

文件存储空间的管理方法：

- 空闲分区表
- 空闲链表法
- 位示图

- 成组链接法

1.空闲分区表：

- 为外存上的所有空闲区建立一张空闲表
- 每个空闲区对应于一个空闲表项
- 其中包括表项序号、该空闲区的第一个盘块号、该区的空闲盘块数等信息

序 号	第一空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	—	—

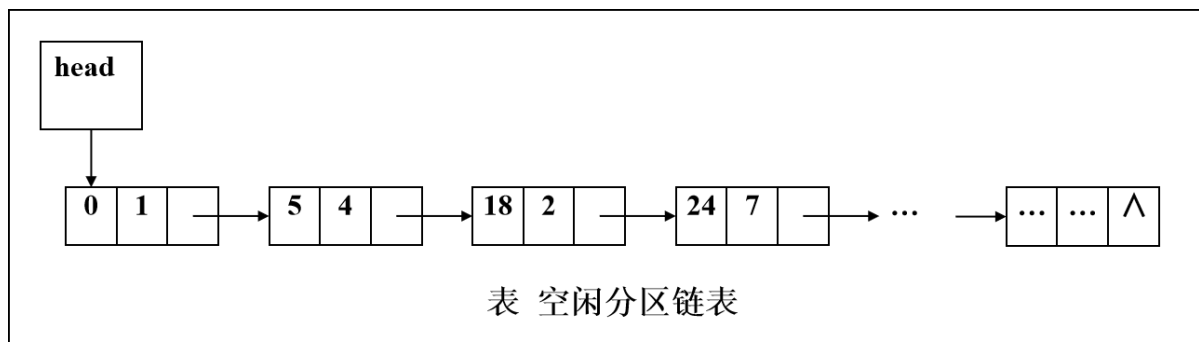
2.空闲链表法：

1) 空闲盘块链

- 将磁盘上的所有空闲空间，以盘块为单位拉成一条链

2) 空闲盘区链

- 将磁盘上的所有空闲盘区(每个盘区可包含若干个盘块)，拉成一条链
- 为了提高对空闲盘区的检索速度，可以采用显式链接方法，亦即，在内存中为空闲盘区建立一张链表
- 每个分区结点内容：起始盘块号、盘块数、指向下一个空闲盘区的指针



3.位示图：

- 利用二进制位0、1表示存储空间中存储块的使用状态
- 空闲分区:0，已分配分区:1（或者相反）
- 磁盘上的所有盘块都有一个二进制位与之对应
- 通常可用 $m \times n$ 个数来构成位示图，并使 $m \times n$ 等于磁盘的总块数

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	1	1	0	0	0	1	1	1	0	0	1	0	0	1	1	0
2	0	0	0	1	1	1	1	1	1	0	0	0	0	1	1	1
3	1	1	1	0	0	0	1	1	1	1	1	1	0	0	0	0
4																
⋮																
16																

分配：扫描、找到为“0”的，转换成相应的盘块号 $b = n(i - 1) + j$ ，修改位示图

回收： $i = (b - 1) \text{DIV } n + 1$ ， $j = (b - 1) \text{MOD } n + 1$ ，修改位示图

(注意，是从0还是从1开始编号)

4.成组链接法

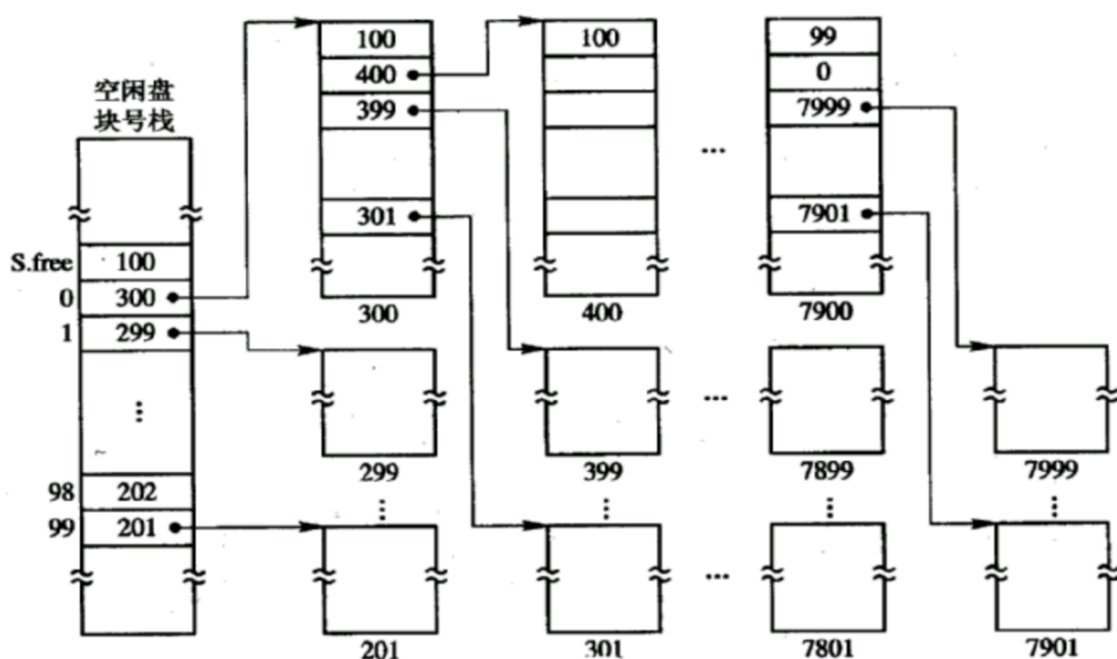


图 6 - 22 空闲盘块的成组链接法

- 100个盘块一组
- 每组合有的盘块总数n和该组所有盘块号记入其前一组的第一个盘块中
- 最末一组只有99个盘块

6.4 文件目录

文件控制块（FCB）：用于描述和控制文件的数据结构

文件目录：文件控制块的有序集合。

1.文件控制块的内容：

- 基本信息：文件名、文件类型等；
- 地址信息：卷（存储文件的设备）、起始地址（起始物理地址）、文件长度（以字节、字或块为单位）等。
- 访问控制信息：文件所有者、访问信息（用户名和口令等）、合法操作等；
- 使用信息：创建时间、创建者身份、当前状态、最近修改时间、最近访问时间等。

2.目录项的两种组织方式：

- FCB存储全部目录内容
- FCB只存储部分目录信息，如文件名和索引节点指针，其余部分保存在索引节点（i节点）。打开文件时将索引节点从磁盘读到内存中。（这样，一块中就可以存多个文件的目录信息，提高文件检索速度）

目录结构

1.单级目录结构

- 所有用户的全部文件目录保存在一张目录表中
- 每个文件的目录项占用一个表项

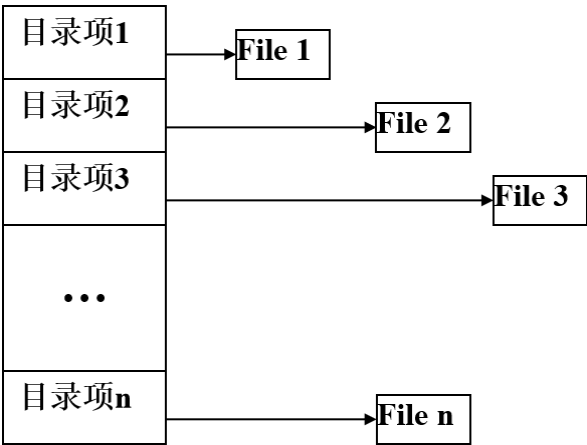


图 单级目录结构

查找速度慢、不允许重名、不便于实现文件共享

2.两级目录结构：

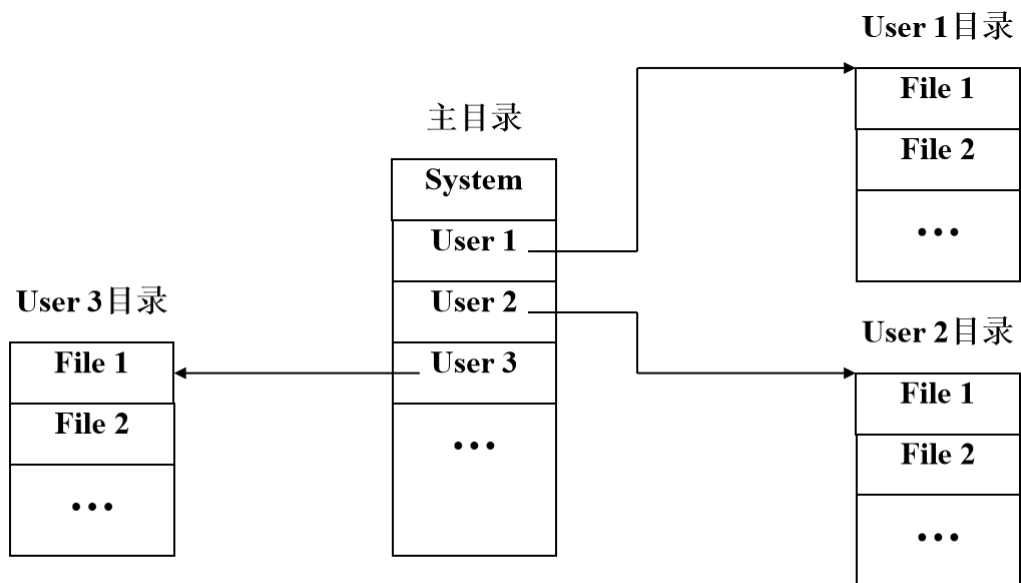
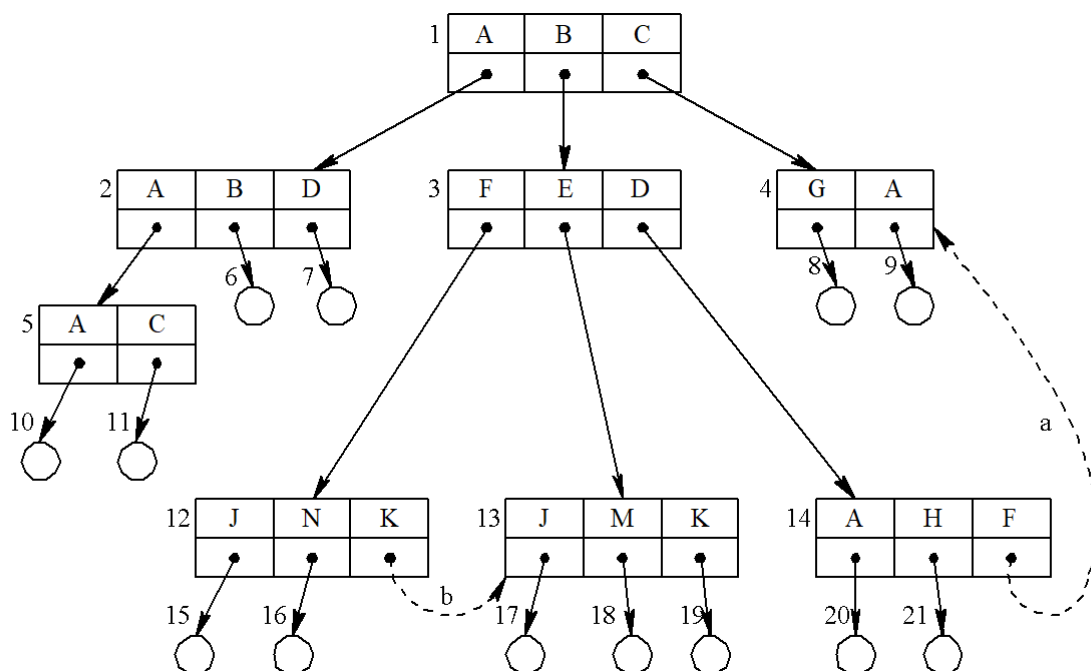


图 5.2 两级目录结构

一定程度解决了重名问题、提高了文件目录检索效率、简单的文件共享

3.多级目录结构

- 又称为树型目录结构
- 主目录在这里被称为根目录
- 把数据文件称为树叶
- 其它的目录均作为树的结点



- 系统中的每一个文件都有唯一的路径名
- 为每个进程设置一个“当前目录”，又称为“工作目录”进程对各文件的访问都相对于“当前目录”而进行

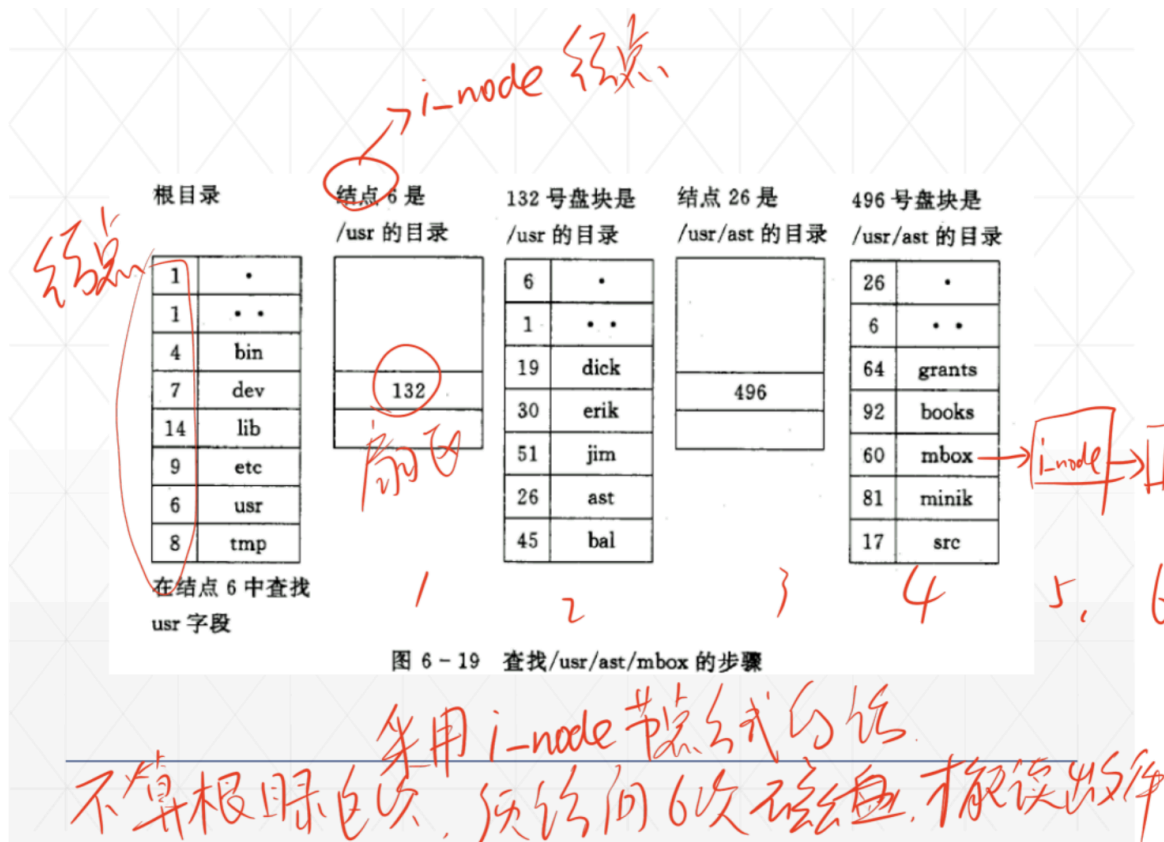
目录查询技术

对目录进行查询的方式有两种：

- 线性检索法
- Hash方法

1.线性检索法：

- 在单级目录中，利用用户提供的文件名，用顺序查找法直接从文件目录中找到指名文件的目录项
- 在树型目录中，用户提供的文件名是由多个文件分量名组成的路径名，此时须对多级目录进行查找



2.Hash方法：

- 建立一张Hash索引文件目录
- 系统利用用户提供的文件名并将它变换为文件目录的索引值
- 再利用该索引值到目录中去查找
- Hash方法将显著地提高检索速度

3.Hash查找过程：

- 在利用Hash值查找目录时，如果目录表中相应的目录项是空的，则表示系统中并无指定文件
- 如果目录项中的文件名与指定文件名相匹配，则表示该目录项正是所要寻找的文件所对应的目录项，故而可从中找到该文件所在的物理地址
- 如果在目录表的相应目录项中的文件名与指定文件名并不匹配，则表示发生了“Hash冲突”。

6.5 文件共享和访问控制

1.文件共享的有效控制涉及两个方面：

- 同时存取 (Simultaneous Access)
- 存取权限 (Access Rights)

2.控制同时存取：

- 允许多个用户同时读文件内容，但不允许同时修改
- 共享用户之一修改文件内容时，可以将整个文件作为临界资源，锁定整个文件
- 也可以仅仅锁定指定的一条记录，允许其他共享用户读/写该文件的其它记录。

3.控制存取权限

- 控制授权用户以合法的方式访问文件
- 执行、读、追加、更新、更改权限、删除

文件共享

实现文件共享的实质就是可以从不同地方打开同一个文件

实现文件共享的方式：

- 利用链接目录项实现法
- 利用索引节点实现法
- 利用符号链实现法

1.链接目录项实现法

- 文件目录项中设置一个链接指针，用于指向共享文件的目录项
- 访问文件时，根据链接指针内容找到共享文件的目录项，读取该目录项中文件起始位置等信息，操作该文件
- 每当有用户（进程）共享文件时，共享文件目录项中的“共享计数”加1；当用户不再共享该文件，撤销链接指针时，“共享计数”减1。
- 只有当共享文件用户数为1时，才能删除共享文件

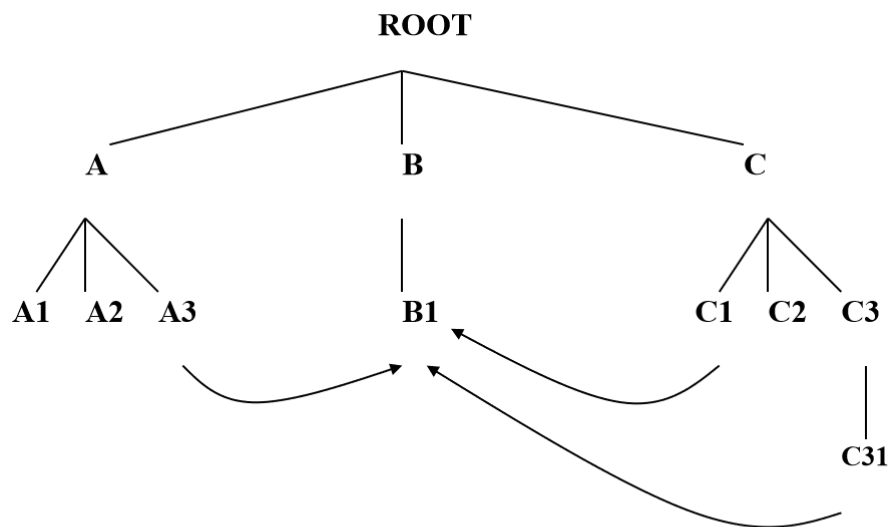


图 链接目录项实现文件共享

2.利用索引节点实现文件共享

- 文件的物理地址及其它的文件属性等信息，不再是放在目录项中，而是放在索引结点中
- 在文件目录中只设置文件名及指向相应索引结点的指针
- 通过共享文件索引节点来共享文件
- 当用户需要共享文件时，在自己的文件目录中新建一个目录项，为共享文件命名(也可用原名)，并将索引节点指针指向共享文件的索引节点

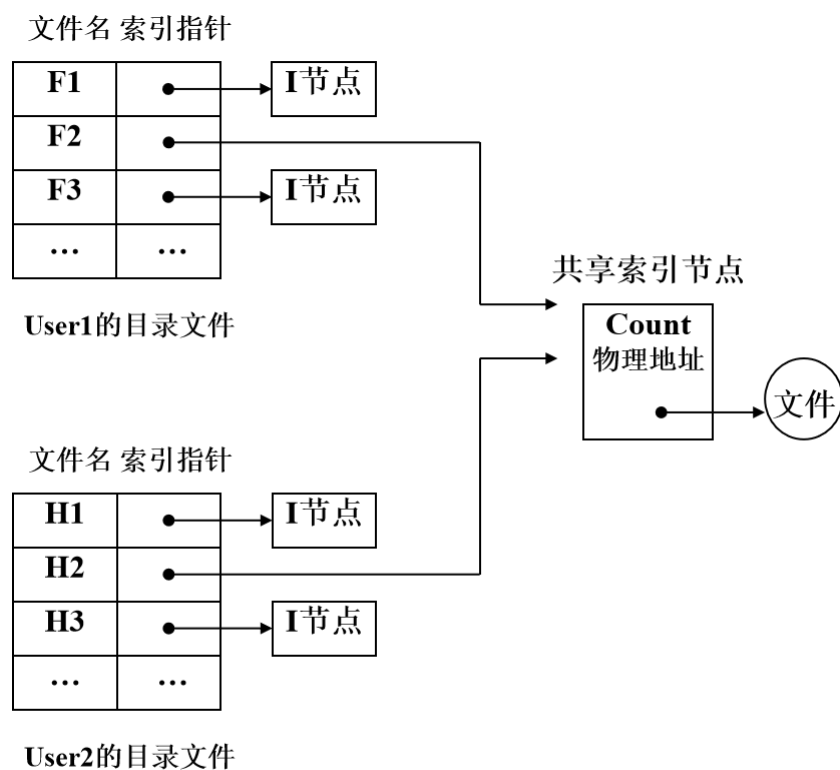


图 利用索引节点实现文件共享



- 在索引结点中还应有一个链接计数count，用于表示链接到本索引结点(亦即文件)上的用户目录项的数目。

- 当用户C创建一个新文件时，他便是该文件的所有者，此时将count置1。
- 当有用户B要共享此文件时，在用户B的目录中增加一目录项，并设置一指针指向该文件的索引结点，此时，文件主仍然是C，count=2。

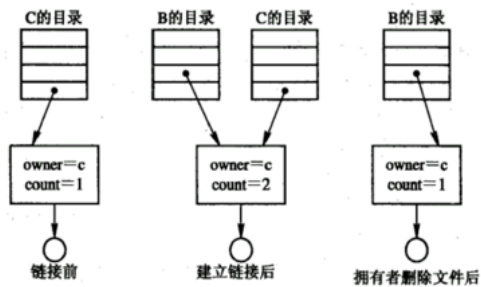


图 6-25 进程 B 链接前后的情况

- 如果用户C不再需要此文件，是否能将此文件删除呢？

电子科技大学

3.利用符号链实现文件共享

- 为使B能共享C的一个文件F，可以由系统创建一个LINK类型的新文件
- 也取名为F并将F写入B的目录中，以实现B的目录与文件F的链接
- 在新文件中只包含被创文件F的路径名
- 读LINK类新文件时，将被OS截获，OS根据新文件中的路径名去读该文件

文件保护

- 不同对象允许实施的操作各不相同
- 系统为所有对象设置一个允许进程实施操作的操作集
- 任何对对象的操作必须符合操作集中的规定，防止未授权进程访问对象

第七章 UnixLinux系统入门（不考）

简单键盘命令：

- (1) `date`: 输出或设置系统日期和时间。
- (2) `who`: 查看系统中所有已登录用户的状况。
- (3) `passwd`: 修改用户密码。
- (4) `logout`、`login`: 登录 Shell 的登录和注销命令。
- (5) `pwd`: 输出工作路径。
- (6) `more`、`less`、`head`、`tail`: 显示或部分显示文件内容。
- (7) `lp/lpstat/cancel`、`lpr/lpq/lprm`: 输出文件。
- (8) `chmod u+x`: 更改文件权限。
- (9) `rm -fr dir`: 删除非空目录。
- (10) `cp -R dir`: 复制目录。
- (11) `fg jobid`: 可以将一个后台进程放到前台。
- (12) `kill`: 删除执行中的程序或工作。
- (13) `ps`: 列出当前运行的进程。

第八章 shell的交互功能与shell程序设计

..... (看PPT, 读读Shell程序)